



HD19 DA Dang Tung Long - Đồ án tốt nghiệp tham khảo của Long Đặng idol K60

Đồ án tốt nghiệp kỹ sư (Trường Đại học Bách khoa Hà Nội)



messages.pdf_cover_qr_code_label

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN ĐIỆN TỬ - VIỄN THÔNG



ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**THIẾT KẾ NEURAL NETWORK TRÊN
FPGA ĐỂ NHẬN DẠNG CHỮ SỐ VIẾT TAY**

Sinh viên thực hiện: **ĐẶNG TÙNG LONG**

Lớp ĐT2 – K59

Giảng viên hướng dẫn: **ThS. NGUYỄN THỊ KIM THOA**

Hà Nội, 06-2019

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
VIỆN ĐIỆN TỬ - VIỄN THÔNG



ĐỒ ÁN
TỐT NGHIỆP ĐẠI HỌC

Đề tài:

**THIẾT KẾ NEURAL NETWORK TRÊN
FPGA ĐỂ NHẬN DẠNG CHỮ SỐ VIẾT TAY**

Sinh viên thực hiện: **ĐẶNG TÙNG LONG**

Lớp ĐT2 – K59

Giảng viên hướng dẫn: **ThS. NGUYỄN THỊ KIM THOA**

Cán bộ phản biện:

Hà Nội, 06-2019

ĐÁNH GIÁ QUYỀN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho giảng viên hướng dẫn)

Tên giảng viên đánh giá:.....

Họ và tên Sinh viên:.....MSSV:.....

Tên đồ án:.....

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém (2); Đạt (3); Giỏi (4); Xuất sắc (5)

Có sự kết hợp giữa lý thuyết và thực hành (20)					
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4 5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4 5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4 5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4 5
Có khả năng phân tích và đánh giá kết quả (15)					
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4 5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng.	1	2	3	4 5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai.	1	2	3	4 5
Kỹ năng viết quyền đồ án (10)					
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy v.v), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4 5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4 5
Thành tựu nghiên cứu khoa học (5) (chọn 1 trong 3 trường hợp)					
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/đạt giải SVNC khoa học giải 3 cấp Viện trở lên/các giải thưởng khoa học (quốc tế/trong nước) từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế	5			
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị sinh viên nghiên cứu khoa học nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành như TI contest.	2			
10c	Không có thành tích về nghiên cứu khoa học	0			
Điểm tổng		/50			
Điểm tổng quy đổi về thang 10					

Nhận xét khác (về thái độ và tinh thần làm việc của sinh viên)

.....

.....

.....

.....

.....

.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

ĐÁNH GIÁ QUYỀN ĐỒ ÁN TỐT NGHIỆP

(Dùng cho cán bộ phản biện)

Giảng viên đánh giá:.....

Họ và tên sinh viên:..... MSSV:.....

Tên đồ án:.....

Chọn các mức điểm phù hợp cho sinh viên trình bày theo các tiêu chí dưới đây:

Rất kém (1); Kém (2); Đạt (3); Giỏi (4); Xuất sắc (5)

Có sự kết hợp giữa lý thuyết và thực hành (20)					
1	Nêu rõ tính cấp thiết và quan trọng của đề tài, các vấn đề và các giả thuyết (bao gồm mục đích và tính phù hợp) cũng như phạm vi ứng dụng của đồ án	1	2	3	4 5
2	Cập nhật kết quả nghiên cứu gần đây nhất (trong nước/quốc tế)	1	2	3	4 5
3	Nêu rõ và chi tiết phương pháp nghiên cứu/giải quyết vấn đề	1	2	3	4 5
4	Có kết quả mô phỏng/thực nghiệm và trình bày rõ ràng kết quả đạt được	1	2	3	4 5
Có khả năng phân tích và đánh giá kết quả (15)					
5	Kế hoạch làm việc rõ ràng bao gồm mục tiêu và phương pháp thực hiện dựa trên kết quả nghiên cứu lý thuyết một cách có hệ thống	1	2	3	4 5
6	Kết quả được trình bày một cách logic và dễ hiểu, tất cả kết quả đều được phân tích và đánh giá thỏa đáng.	1	2	3	4 5
7	Trong phần kết luận, tác giả chỉ rõ sự khác biệt (nếu có) giữa kết quả đạt được và mục tiêu ban đầu đề ra đồng thời cung cấp lập luận để đề xuất hướng giải quyết có thể thực hiện trong tương lai.	1	2	3	4 5
Kỹ năng viết quyền đồ án (10)					
8	Đồ án trình bày đúng mẫu quy định với cấu trúc các chương logic và đẹp mắt (bảng biểu, hình ảnh rõ ràng, có tiêu đề, được đánh số thứ tự và được giải thích hay đề cập đến trong đồ án, có căn lề, dấu cách sau dấu chấm, dấu phẩy v.v), có mở đầu chương và kết luận chương, có liệt kê tài liệu tham khảo và có trích dẫn đúng quy định	1	2	3	4 5
9	Kỹ năng viết xuất sắc (cấu trúc câu chuẩn, văn phong khoa học, lập luận logic và có cơ sở, từ vựng sử dụng phù hợp v.v.)	1	2	3	4 5
Thành tựu nghiên cứu khoa học (5) (chọn 1 trong 3 trường hợp)					
10a	Có bài báo khoa học được đăng hoặc chấp nhận đăng/đạt giải SVNC khoa học giải 3 cấp Viện trở lên/các giải thưởng khoa học (quốc tế/trong nước) từ giải 3 trở lên/ Có đăng ký bằng phát minh sáng chế	5			
10b	Được báo cáo tại hội đồng cấp Viện trong hội nghị sinh viên nghiên cứu khoa học nhưng không đạt giải từ giải 3 trở lên/Đạt giải khuyến khích trong các kỳ thi quốc gia và quốc tế khác về chuyên ngành như TI contest.	2			
10c	Không có thành tích về nghiên cứu khoa học	0			
Điểm tổng		/50			
Điểm tổng quy đổi về thang 10					

Nhận xét khác của cán bộ phản biện

.....

.....

.....

.....

.....

.....

Ngày: ... / ... / 20...

Người nhận xét

(Ký và ghi rõ họ tên)

LỜI NÓI ĐẦU

Trước khi trình bày nội dung phần báo cáo đồ án của mình, em xin gửi lời cảm ơn chân thành nhất tới Th.S Nguyễn Thị Kim Thoa, người đã trực tiếp hướng dẫn, cung cấp các tài liệu cho em trong suốt quá trình thực hiện đồ án. Em cũng xin được gửi lời cảm ơn chân thành đến Lab ESRC đã cho em mượn thiết bị để hoàn thành đồ án này.

Do thời gian có hạn và hạn chế về mặt kiến thức, báo cáo không tránh khỏi một vài sai sót nhỏ. Vì vậy, em rất mong nhận được ý kiến đóng góp của các thầy cô giáo và các bạn để đề tài được hoàn thiện hơn.

Hà Nội, tháng 06 năm 2019

Sinh viên thực hiện

Đặng Tùng Long

LỜI CAM ĐOAN

Tôi là Đặng Tùng Long, mã số sinh viên 20142642, sinh viên lớp Điện tử 2, khóa K59. Người hướng dẫn là Th.S. Nguyễn Thị Kim Thoa. Tôi xin cam đoan toàn bộ nội dung được trình bày trong đề án *Thiết kế neural network trên FPGA để nhận dạng chữ số viết tay* là kết quả quá trình tìm hiểu và nghiên cứu của tôi. Các dữ liệu được nêu trong đề án là hoàn toàn trung thực, phản ánh đúng kết quả đo đạc thực tế. Mọi thông tin trích dẫn đều tuân thủ các quy định về sở hữu trí tuệ; các tài liệu tham khảo được liệt kê rõ ràng. Tôi xin chịu hoàn toàn trách nhiệm với những nội dung được viết trong đề án này.

Hà Nội, tháng 06 năm 2019

Người cam đoan

Đặng Tùng Long

MỤC LỤC

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT.....	i
DANH MỤC HÌNH VẼ.....	ii
DANH MỤC BẢNG BIỂU.....	iii
TÓM TẮT ĐỒ ÁN.....	iv
PHẦN MỞ ĐẦU.....	1
CHƯƠNG 1. GIỚI THIỆU.....	2
1.1. Giới thiệu về Artificial Neural Network.....	2
1.1.1. Lịch sử hình thành và phát triển.....	2
1.1.2. Nguyên lý hoạt động.....	3
1.1.3. Thuật toán Gradient Descent.....	9
1.2. Giới thiệu về FPGA.....	10
1.2.1. Khái niệm FPGA.....	10
1.2.2. Schematic-based flow.....	11
1.2.3. HDL-based flow.....	13
1.4. Giới thiệu về SystemVerilog.....	14
1.5. Giới thiệu về mô hình FSM.....	14
1.6. Giới thiệu về kiểm chứng thiết kế.....	16
1.7. Kết luận chương.....	17
CHƯƠNG 2. THIẾT KẾ NEURAL NETWORK VỚI MATLAB.....	18
2.1. Thuật toán thiết kế.....	18
2.2. Huấn luyện trên Matlab.....	19
2.2.1. Chuẩn bị cơ sở dữ liệu.....	19
2.2.2. Tiền xử lý huấn luyện.....	20
2.2.3. Huấn luyện Neural Network.....	21
2.3. Kiểm tra trên Matlab.....	25
2.3.1. Chuẩn bị cơ sở dữ liệu.....	25
2.3.2. Tiền xử lý kiểm tra.....	25
2.3.3. Kiểm tra Neural Network.....	26

2.4. Kết luận chương.....	27
CHƯƠNG 3. THIẾT KẾ NEURAL NETWORK TRÊN FPGA.....	28
3.1. Yêu cầu kỹ thuật.....	28
3.1.1. Đầu vào và đầu ra.....	28
3.1.2. Hoạt động.....	29
3.2. Kế hoạch kiểm chứng.....	31
3.3. Thiết kế RTL.....	32
3.3.1. Khối DUT.....	32
3.3.2. Khối ann.....	34
3.3.3. Biểu diễn tham số của Neural Network trên FPGA.....	45
3.4. Thiết kế testbench.....	46
3.4.1. Kiến trúc của testbench.....	46
3.4.2. Kết quả kiểm chứng thiết kế.....	49
3.5. Kết luận chương.....	51
KẾT LUẬN.....	52
TÀI LIỆU THAM KHẢO.....	53
BẢNG ĐỐI CHIẾU THUẬT NGỮ ANH VIỆT.....	54

DANH MỤC KÝ HIỆU VÀ CHỮ VIẾT TẮT

Chữ viết tắt	Cụm từ viết đầy đủ
ANN	Artificial Neural Network
FPGA	Field Programable Gate Array
OTP	One-time Programmable
ISP	In-system Programmable
CLB	Configurable Logic Block
LUT	Look Up Table
HDL	Hardware Description Language
RTL	Register Transfer Level
DUT	Design Under Test
I/O	Input/Output

DANH MỤC HÌNH VẼ

Hình 1.1 Cấu trúc 3 lớp [2].....	4
Hình 1.2 Cấu trúc 3 lớp với các tham số [2].....	5
Hình 1.3 Khảo sát sự biến thiên của một đa thức bậc 2 một biến [3].....	9
Hình 1.4 Thiết kế FPGA theo schematic-based flow [4].....	11
Hình 1.5 Mapping các cổng logic vào LUT [4].....	12
Hình 1.6 Thiết kế FPGA theo HDL-based flow [4].....	13
Hình 1.7 Sơ đồ khối của FSMD [6].....	15
Hình 1.8 Mô hình testbench đủ các lớp [7].....	16
Hình 2.1 Lưu đồ thuật toán thiết kế trên Matlab.....	18
Hình 2.2 Cơ sở dữ liệu MNIST.....	20
Hình 2.3 Hàm softmax với vector 4 phần tử.....	23
Hình 2.4 Hiệu suất huấn luyện.....	24
Hình 3.1 Các chân I/O của DUT.....	28
Hình 3.2 Hoạt động ghi vào bộ nhớ.....	29
Hình 3.3 Hoạt động nhận dạng chữ số.....	30
Hình 3.4 Khối DUT.....	32
Hình 3.5 Ghi vào và đọc ra với single_port_ram_with_init.....	33
Hình 3.6 Khối ann.....	35
Hình 3.7 Khối ann_dp.....	36
Hình 3.8 Hàm tansig thực sự và hàm tansig đã tuyến tính hóa.....	39
Hình 3.9 Sơ đồ FSM của ann_fsm.....	42
Hình 3.10 Sơ đồ FSM của ann_fsm (tiếp tục).....	43
Hình 3.11 Kết quả biểu diễn số 0.15625 [9].....	45
Hình 3.12 Sơ đồ khối testbench.....	47
Hình 3.13 Biên dịch thành công cho kit EP2C35F672C6.....	50

DANH MỤC BẢNG BIỂU

Bảng 2.1 Mã hóa giám sát viên theo chữ số.....	22
Bảng 2.2 Kết quả kiểm tra Neural Network trên Matlab.....	27
Bảng 3.1 Chức năng của các chân I/O.....	28
Bảng 3.2 Chữ số biểu thị ứng với đầu ra.....	30
Bảng 3.3 Ý nghĩa các dải địa chỉ trong ram.....	33
Bảng 3.4 Các chân I/O của single_port_ram_with_init.....	33
Bảng 3.5 Các chân I/O của ann.....	34
Bảng 3.6 Các tín hiệu control signal trong khối ann.....	40
Bảng 3.7 Các tín hiệu internal status trong khối ann.....	41
Bảng 3.8 Giá trị đầu ra bằng 1 tại mỗi trạng thái của ann_fsm.....	44
Bảng 3.9 Chức năng các khối trong testbench.....	48
Bảng 3.10 Kết quả kiểm chứng thiết kế.....	49
Bảng 3.11 Trễ xử lý trung bình với từng chữ số.....	50

TÓM TẮT ĐỒ ÁN

Trong đồ án này, trước hết, em triển khai Neural Network trên Matlab để nghiên cứu về độ chính xác cũng như tính khả thi trong việc triển khai trên thiết bị FPGA là kit Altera DE2 – EP2C35F672C6. Sau khi triển khai thành công trên Matlab, em đã tìm cách tối ưu hóa về kích thước và độ chính xác để có thể triển khai trên thiết bị FPGA đó. Thiết kế trên FPGA nhận đầu vào là vector chứa các điểm ảnh của ảnh đã qua tiền xử lý, và chỉ làm nhiệm vụ nhận dạng chữ số. Đầu ra là vector thể hiện chữ số nào đang được biểu diễn trên ảnh. Kết quả thu được là độ chính xác trên tập kiểm tra MNIST đạt 94.02%.

Việc huấn luyện trên Matlab đã được thực hiện nhiều lần, trước mỗi lần huấn luyện thì kích thước của Neural Network được giảm xuống. Mục tiêu của huấn luyện trên Matlab là để tìm ra mạng có kích thước nhỏ nhất có thể mà vẫn đảm bảo được độ chính xác trên 90%.

Sau khi huấn luyện thành công trên Matlab thì Neural Network đó được triển khai trên FPGA với những yêu cầu kỹ thuật cho phần cứng. Từ yêu cầu kỹ thuật, em xây dựng kế hoạch kiểm chứng và tuân theo nó để kiểm chứng thiết kế. Thiết kế đã đạt yêu cầu kỹ thuật khi đã đạt được độ bao phủ chức năng là 100%. Ngoài ra, em còn thống kê trễ xử lý trung bình khi thực hiện nhận dạng từng loại chữ số, từ đó rút ra được một số nhận xét về tính khả thi khi sử dụng thiết kế này trong một hệ thống nhận dạng theo thời gian thực.

ABSTRACT

In this project, I built many neural networks on Matlab to find the most suitable pattern to implement it on DE2 – EP2C35F672C6 board, which is a FPGA device produced by Altera. The input of the design is a vector containing pixels of a preprocessed photo. The design is responsible for digit recognition with the photo. The output of the design indicates which digit is being displayed on the photo. The result is that the accuracy rate is 94.02% with MNIST test database.

I trained the Neural Network on Matlab many times. Before each time, the size of the net is reduced. The goal of the training process on Matlab is to find the smallest net which can recognize the MNIST test database with accuracy rate larger than 90%.

After the successful training process on Matlab, the network was implemented on FPGA with specification. According to the specification, I built verification plan and followed it to verify my design. The design met all specification with 100% of functional coverage. In addition, I made statistics for average latency of each group of digit to consider whether my design could be feasible to used in a real-time system.

PHẦN MỞ ĐẦU

Trong thời gian học tập tại Đại học Bách Khoa Hà Nội, FPGA và các ngôn ngữ mô tả phần cứng là lĩnh vực mà em yêu thích và giành khá nhiều thời gian để nghiên cứu và thực hành. Nhưng hiện nay, trí tuệ nhân tạo mới là cụm từ mà người ta đang nhắc tới nhiều. Trước xu hướng chung của xã hội, em đã trang bị cho mình được một số kiến thức về trí tuệ nhân tạo và một số ứng dụng của nó trong đời sống. Qua các bài viết trên Internet, em đã biết đến Neural Network là một phần nhỏ trong trí tuệ nhân tạo và người ta có thể sử dụng nó để nhận dạng vật thể. Từ đó, em nảy ra ý tưởng thiết kế hệ thống nhận dạng chữ số viết tay trên FPGA.

Báo cáo đồ án được chia làm 3 chương chính như sau:

Chương 1 giới thiệu về Neural Network, FPGA, ngôn ngữ SystemVerilog, mô hình FSM và kiểm chứng thiết kế.

Chương 2 trình bày về quá trình huấn luyện và kiểm tra Neural Network trên Matlab với cơ sở lý thuyết đã trình bày trong Chương 1.

Chương 3 trình bày về việc thiết kế Neural Network trên FGPA, bao gồm: yêu cầu kỹ thuật, kế hoạch kiểm chứng, thiết kế RTL và thiết kế testbench. Toàn bộ việc thiết kế đều tuân theo cơ sở lý thuyết đã trình bày trong Chương 1. Kết quả của thiết kế đạt yêu cầu kỹ thuật như thế nào đã được trình bày trong chương này.

CHƯƠNG 1. GIỚI THIỆU

Trong chương này, các kiến thức tổng quan về Artificial Neural Network, FPGA, FSM, và kiểm chứng thiết kế sẽ được trình bày.

1.1. Giới thiệu về Artificial Neural Network

1.1.1. Lịch sử hình thành và phát triển

Artificial Neural Network (viết tắt là ANN), thường được gọi tắt Neural Network, là mạng neuron nhân tạo. Neural Network cũng giống như các lĩnh vực khác của khoa học, có lịch sử phát triển trải qua nhiều thăng trầm. Trong cuốn sách online *A brief introduction to neural network* [1] tác giả đã liệt kê ra một số mốc thời gian trong lịch sử hình thành và phát triển của nó.

Neural Network ra đời từ khoảng đầu những năm 1940 và gần như cùng thời điểm với sự ra đời của việc lập trình máy tính điện tử. Năm 1943, Warren McCulloch và Walter Pitts đã giới thiệu mô hình mạng neuron và cho thấy loại mạng đơn giản này có thể tính được gần chính xác các hàm logic và toán học. Hơn nữa, với sự giúp đỡ của Konrad Zuse, tiền thân của thiết bị điện tử sử dụng Neural Network cũng được bắt đầu phát triển [1].

Năm 1949, Donald O. Hebb xây dựng công thức luật Hebbian, là luật khái quát về quy tắc cơ bản cho quá trình học tập của mạng neuron. Quy tắc này ngụ ý rằng, liên kết giữa hai neuron được củng cố khi cả hai neuron được kích hoạt cùng một lúc. Hebb đã tin là quy tắc này đúng, nhưng vì không có công trình nghiên cứu về mạng neuron nên ông không thể chứng minh nó [1].

Năm 1957 – 1958, tại MIT, Frank Rosenblatt, Charles Wightman và các đồng nghiệp đã chế tạo thành công máy tính sử dụng mạng neuron, gọi là Mark I perceptron, có khả năng nhận dạng các số đơn giản bằng cảm biến hình ảnh 20x20 điểm ảnh (pixel), với cơ chế là 512 chiết áp điều khiển động cơ, mỗi chiết áp đại diện cho một tham số thay đổi được của mạng [1].

Năm 1959, Frank Rosenblatt đã mô tả các phiên bản khác nhau của perceptron trong mạng neuron, xây dựng công thức và chứng minh được định lý gọi là perceptron convergence theorem. Ông đã mô phỏng các lớp neuron bất chước võng mạc, các ngưỡng chuyển đổi và một nguyên tắc học tập để điều chỉnh các tham số của các kết nối trong mạng. [1]

Năm 1965, trong cuốn sách Machine Learning của mình, Nils Nilsson đã đưa ra cái nhìn tổng quát về sự tiến triển và thành quả của các công trình nghiên cứu về ANN

trong thời kì này. Nó được coi là lý thuyết cơ bản cho việc tự học của các trí thông minh nhân tạo [1].

Khoảng thời gian tiếp theo là khoảng thời gian trầm lắng đối với việc nghiên cứu về Neural Network, bởi tài trợ cho các công trình nghiên cứu này còn hạn chế. Các công bố và hội thảo về lĩnh vực này có ít, chỉ có các nhà nghiên cứu hoạt động riêng lẻ và không có sự trao đổi lẫn nhau [1].

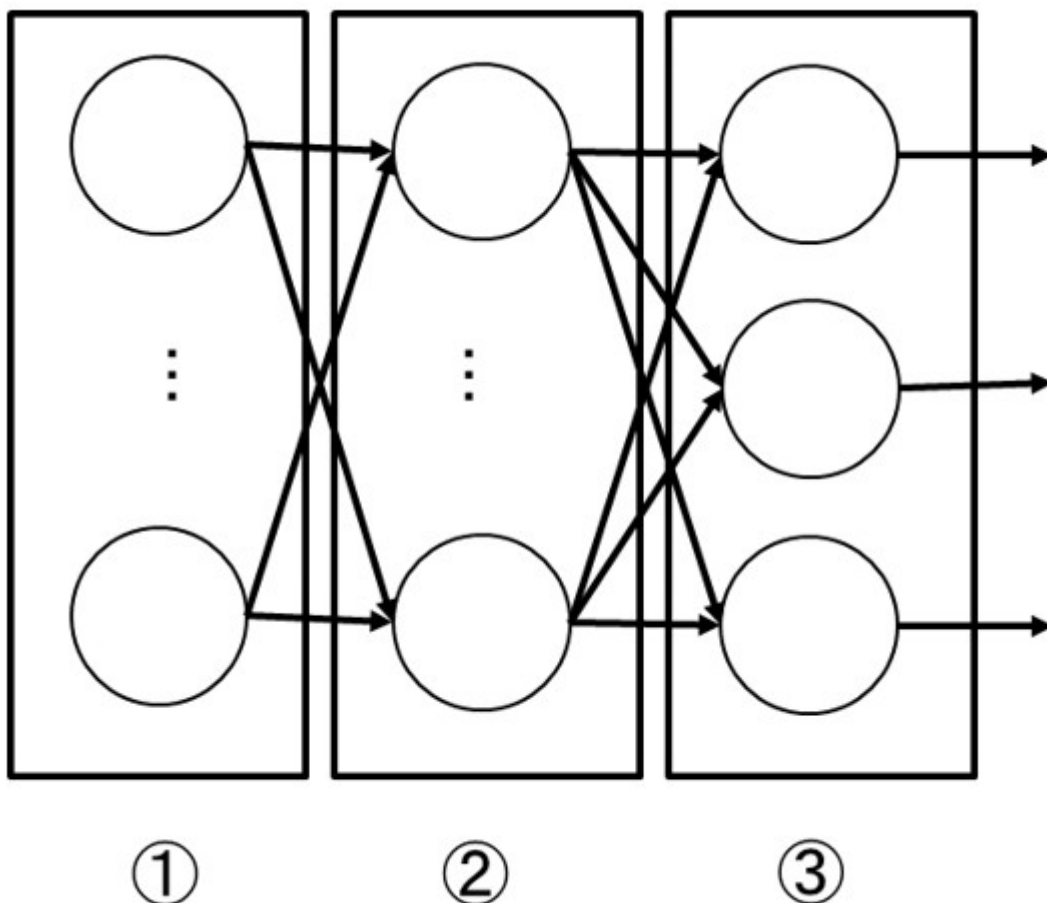
Đến năm 1985, Neural Network bước vào thời kỳ phục hưng khi John Hopfield tìm được lời giải chấp nhận được cho bài toán Travelling Salesman Problem (TSP) sử dụng mạng Hopfield. Nội dung của TSP là: cho danh sách các thành phố và khoảng cách giữa mỗi cặp thành phố, từ thành phố gốc ta cần tìm con đường ngắn nhất có thể đến thăm từng thành phố và trở về thành phố gốc. Đây là một bài toán quan trọng trong lĩnh vực khoa học máy tính [1].

Năm 1986, quy tắc học lan truyền ngược của mạng neuron được phát triển và xuất bản rộng rãi. Từ đó đến nay, nghiên cứu về Neural Network gần như là bùng nổ với nhiều nghiên cứu không thể liệt kê hết [1].

1.1.2. Nguyên lý hoạt động

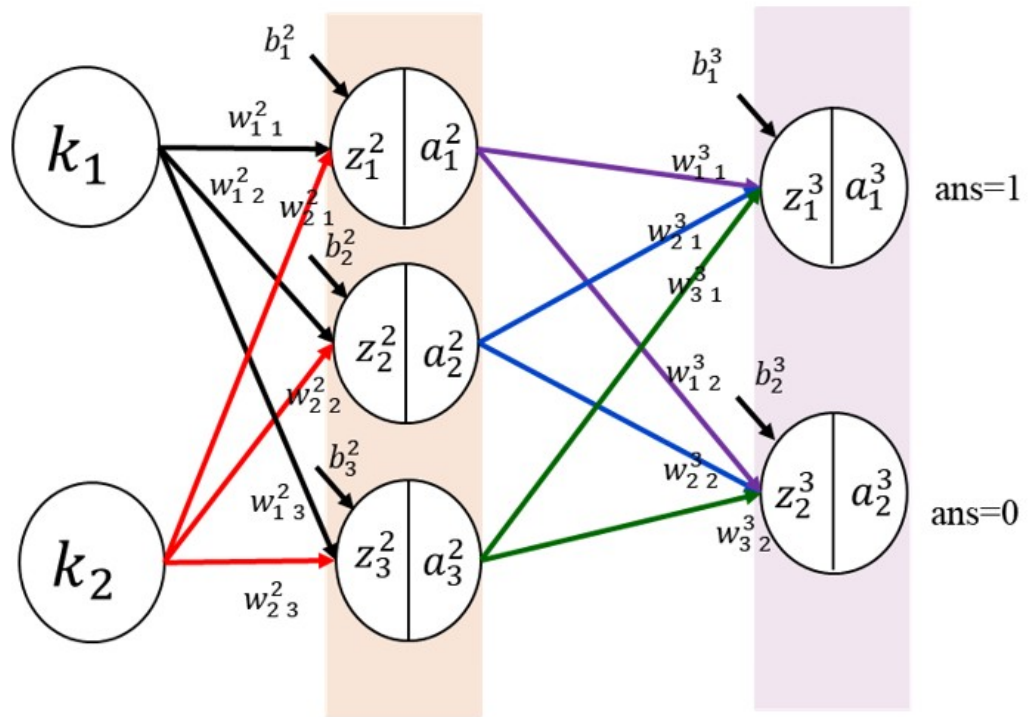
Artificial Neural Network là thuật ngữ chung chỉ các loại mạng neuron nhân tạo. Qua lịch sử phát triển, các nhà nghiên cứu đã công bố rất nhiều loại mạng neuron nhân tạo khác nhau. Trong phần này, nguyên lý hoạt động được trình bày là nguyên lý chung của Neural Network được giới thiệu bởi tổ chức LSI Design Contest [2].

Não người có rất nhiều neuron, nó nhận tín hiệu đầu vào và tạo tín hiệu đầu ra. Ví dụ như khi tay ta chạm vào một vật nóng, thì đầu vào là nóng được truyền tới tủy sống thông qua các tế bào thần kinh bên trong, và nó sẽ đưa ra đầu ra là phản xạ rút tay khỏi vật nóng. Neural Network là một mô hình toán học bắt chước hệ thần kinh của con người. Về cơ bản thì Neural Network có 3 lớp, đó là (1) lớp đầu vào, (2) lớp ẩn và (3) lớp đầu ra được minh họa trên Hình 1.1. Cấu trúc này được gọi là cấu trúc 3 lớp. Đối với những mạng phức tạp hơn thì có thể có nhiều hơn một lớp ẩn [2].



Hình 1.1 Cấu trúc 3 lớp [2]

Tín hiệu được đưa vào ở lớp (1) và đi ra ở lớp (3). Trước khi đưa tín hiệu vào (1) thì tại đầu ra chúng ta có một giám sát viên (supervisor). Giám sát viên chính là đầu ra kỳ vọng của mạng với mục tiêu là điều chỉnh các tham số trong mạng sao cho sai lệch giữa đầu ra thực tế (output) và giám sát viên là nhỏ nhất có thể [2].



Hình 1.2 Cấu trúc 3 lớp với các tham số [2]

Hình 1.2 minh họa một mạng neuron theo cấu trúc 3 lớp với các tham số ứng với từng lớp. Định nghĩa của các tham số trong hình như sau:

k_i là tín hiệu vào tại lớp đầu vào.

w_{ij}^2 là weight nối từ lớp đầu vào đến lớp ẩn.

b_i^2 là bias của lớp ẩn.

z_i^2 là đầu vào của lớp ẩn.

a_i^2 là đầu ra của lớp ẩn.

w_{ij}^3 là weight nối từ lớp ẩn đến lớp đầu ra.

b_i^3 là bias của lớp đầu ra.

z_i^3 là đầu vào của lớp đầu ra.

a_i^3 là đầu ra của lớp đầu ra.

t_i là giám sát viên với $t_1=1$ và $t_2=0$.

Mối liên hệ giữa các tham số trên được biểu diễn theo chiều thuận và chiều nghịch.

Theo chiều thuận, ta coi tín hiệu vào tại lớp đầu vào (k_i), các weight và bias (w_{ij}^2 , b_i^2 , w_{ij}^3 , b_i^3) là các số đã biết, mục tiêu là tính đầu ra a_i^3 của lớp đầu ra theo các công thức (1.1), (1.2), (1.3) và (1.4).

$$\begin{bmatrix} z_1^2 \\ z_2^2 \\ z_3^2 \end{bmatrix} = \begin{bmatrix} w_{11}^2 & w_{21}^2 \\ w_{12}^2 & w_{22}^2 \\ w_{13}^2 & w_{23}^2 \end{bmatrix} \begin{bmatrix} k_1 \\ k_2 \end{bmatrix} + \begin{bmatrix} b_1^2 \\ b_2^2 \\ b_3^2 \end{bmatrix} \quad (1.1)$$

$$\begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix} = \begin{bmatrix} a(z_1^2) \\ a(z_2^2) \\ a(z_3^2) \end{bmatrix} \quad (1.2)$$

$$\begin{bmatrix} z_1^3 \\ z_2^3 \end{bmatrix} = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 \\ w_{12}^3 & w_{22}^3 & w_{32}^3 \end{bmatrix} \begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix} + \begin{bmatrix} b_1^3 \\ b_2^3 \end{bmatrix} \quad (1.3)$$

$$\begin{bmatrix} a_1^3 \\ a_2^3 \end{bmatrix} = \begin{bmatrix} a(z_1^3) \\ a(z_2^3) \end{bmatrix} \quad (1.4)$$

Trong công thức (1.2) và (1.4), chúng ta có thể sử dụng bất kỳ hàm nào có khả năng phân biệt và chuẩn hóa để làm hàm kích hoạt (activate function) [2], ví dụ như dùng hàm tansig thì công thức (1.2) trở thành (1.5).

$$a_i^2 = \text{tansig}(z_i^2) = \frac{2}{1 + e^{-2z_i^2}} - 1 \text{ với } i=1,2,3 \quad (1.5)$$

Đặt:

$$K = \begin{bmatrix} k_1 \\ k_2 \end{bmatrix}$$

$$W_2 = \begin{bmatrix} w_{11}^2 & w_{21}^2 \\ w_{12}^2 & w_{22}^2 \\ w_{13}^2 & w_{23}^2 \end{bmatrix}$$

$$B_2 = \begin{bmatrix} b_1^2 \\ b_2^2 \\ b_3^2 \end{bmatrix}$$

$$Z2 = \begin{bmatrix} z_1^2 \\ z_2^2 \\ z_3^2 \end{bmatrix}$$

$$A2 = \begin{bmatrix} a_1^2 \\ a_2^2 \\ a_3^2 \end{bmatrix}$$

$$W3 = \begin{bmatrix} w_{11}^3 & w_{21}^3 & w_{31}^3 \\ w_{12}^3 & w_{22}^3 & w_{32}^3 \end{bmatrix}$$

$$B3 = \begin{bmatrix} b_1^3 \\ b_2^3 \end{bmatrix}$$

$$Z3 = \begin{bmatrix} z_1^3 \\ z_2^3 \end{bmatrix}$$

$$A3 = \begin{bmatrix} a_1^3 \\ a_2^3 \end{bmatrix}$$

Các công thức (1.1), (1.2), (1.3), (1.4) trở thành:

$$Z2 = W2.K + B2 \quad (1.6)$$

$$A2 = A(Z2) \quad (1.7)$$

$$Z3 = W3.A2 + B3 \quad (1.8)$$

$$A3 = A(Z3) \quad (1.9)$$

Với mỗi vector K thứ n đưa vào lớp đầu vào ta sẽ tính được một hàm lỗi bình phương (square error function) như công thức (1.10)

$$C_n = \frac{1}{2} \{ (a_1^3[n] - t_1[n])^2 + (a_2^3[n] - t_2[n])^2 \} \quad (1.10)$$

Khi đưa n vector vào lớp đầu vào, ta có hàm giá (cost function) C là tổng của các hàm lỗi bình phương đã có, như công thức số (1.11), với C_n tính như công thức (1.10).

$$C = C_1 + C_2 + \dots + C_n \quad (1.11)$$

Từ bây giờ chúng ta bắt đầu xây dựng công thức theo chiều nghịch sử dụng thuật toán lan truyền ngược (backpropagation). Từ các công thức (1.1), (1.2), (1.3), (1.4), (1.5), (1.6), (1.7), (1.8), (1.9), (1.10), ta thấy C là một hàm của các weight và bias ($w_{ij}^2, b_i^2, w_{ij}^3, b_i^3$). Bởi vậy ta có công thức đạo hàm của C theo các weight và bias đó.

Để tính đạo hàm của C theo các weight và bias, ta cần áp dụng quy tắc tính đạo hàm của hàm hợp. Quy tắc đạo hàm của hàm hợp như sau. Với f là hàm của u và v , tức là $f = f(u, v)$; u và v là hàm của x , tức là $u = u(x)$ và $v = v(x)$; nếu $f(u)$, $f(v)$, $u(x)$, $v(x)$ tồn tại đạo hàm thì ta có:

$$\frac{\partial f}{\partial x} = \frac{\partial f}{\partial u} \frac{\partial u}{\partial x} + \frac{\partial f}{\partial v} \frac{\partial v}{\partial x} \quad (1.12)$$

Giả sử ta cần tính đạo hàm của C theo w_{11}^2 , áp dụng quy tắc đạo hàm của hàm hợp ta có:

$$\frac{\partial C}{\partial w_{11}^2} = \frac{\partial C_1}{\partial w_{11}^2} + \frac{\partial C_2}{\partial w_{11}^2} + \dots + \frac{\partial C_n}{\partial w_{11}^2} \quad (1.13)$$

$$\frac{\partial C_n}{\partial w_{11}^2} = \frac{\partial C_n}{\partial a_1^3[n]} \left(\frac{\partial a_1^3[n]}{\partial w_{11}^2} + \frac{\partial a_2^3[n]}{\partial w_{11}^2} \right) = (a_1^3[n] - t_1[n]) \left(\frac{\partial a_1^3[n]}{\partial w_{11}^2} + \frac{\partial a_2^3[n]}{\partial w_{11}^2} \right) \quad (1.14)$$

$$\frac{\partial a_1^3[n]}{\partial w_{11}^2} = \frac{\partial a_1^3[n]}{\partial z_1^3[n]} \frac{\partial z_1^3[n]}{\partial a_1^2[n]} \frac{\partial a_1^2[n]}{\partial z_1^2[n]} \frac{\partial z_1^2[n]}{\partial w_{11}^2} = \frac{\partial a_1^3[n]}{\partial z_1^3[n]} w_{11}^3 \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.15)$$

$$\frac{\partial a_2^3[n]}{\partial w_{11}^2} = \frac{\partial a_2^3[n]}{\partial z_2^3[n]} \frac{\partial z_2^3[n]}{\partial a_1^2[n]} \frac{\partial a_1^2[n]}{\partial z_1^2[n]} \frac{\partial z_1^2[n]}{\partial w_{11}^2} = \frac{\partial a_2^3[n]}{\partial z_2^3[n]} w_{12}^3 \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.16)$$

Từ (1.14), (1.15), (1.16) ta có:

$$\frac{\partial C_n}{\partial w_{11}^2} = (a_1^3[n] - t_1[n]) \left(\frac{\partial a_1^3[n]}{\partial z_1^3[n]} w_{11}^3 + \frac{\partial a_2^3[n]}{\partial z_2^3[n]} w_{12}^3 \right) \frac{\partial a_1^2[n]}{\partial z_1^2[n]} k_1[n] \quad (1.17)$$

Từ công thức (1.13) và (1.17) ta tính được $\frac{\partial C}{\partial w_{11}^2}$

Theo thuật toán gradient descent, ta cập nhật giá trị mới của w_{11}^2 để tối ưu C .

$$(new) w_{11}^2 = (old) w_{11}^2 - \eta \frac{\partial C}{\partial w_{11}^2} \quad (1.18)$$

Trong công thức (1.18), η là một số dương gọi là tốc độ học (learning rate). Chi tiết về thuật toán gradient descent sẽ được trình bày trong phần sau, qua đó sẽ trả lời được câu hỏi rằng tại sao cập nhật w_{11}^2 theo công thức (1.18) sẽ tối ưu được C .

Để tối ưu C thì các weight và bias khác cũng được cập nhật theo cách tương tự.

Sau khi thay đổi, chúng ta lại bắt đầu lại bằng việc đưa K vào và tiếp tục tính toán, cho đến khi đầu ra đạt giá trị gần với giám sát viên nhất [2].

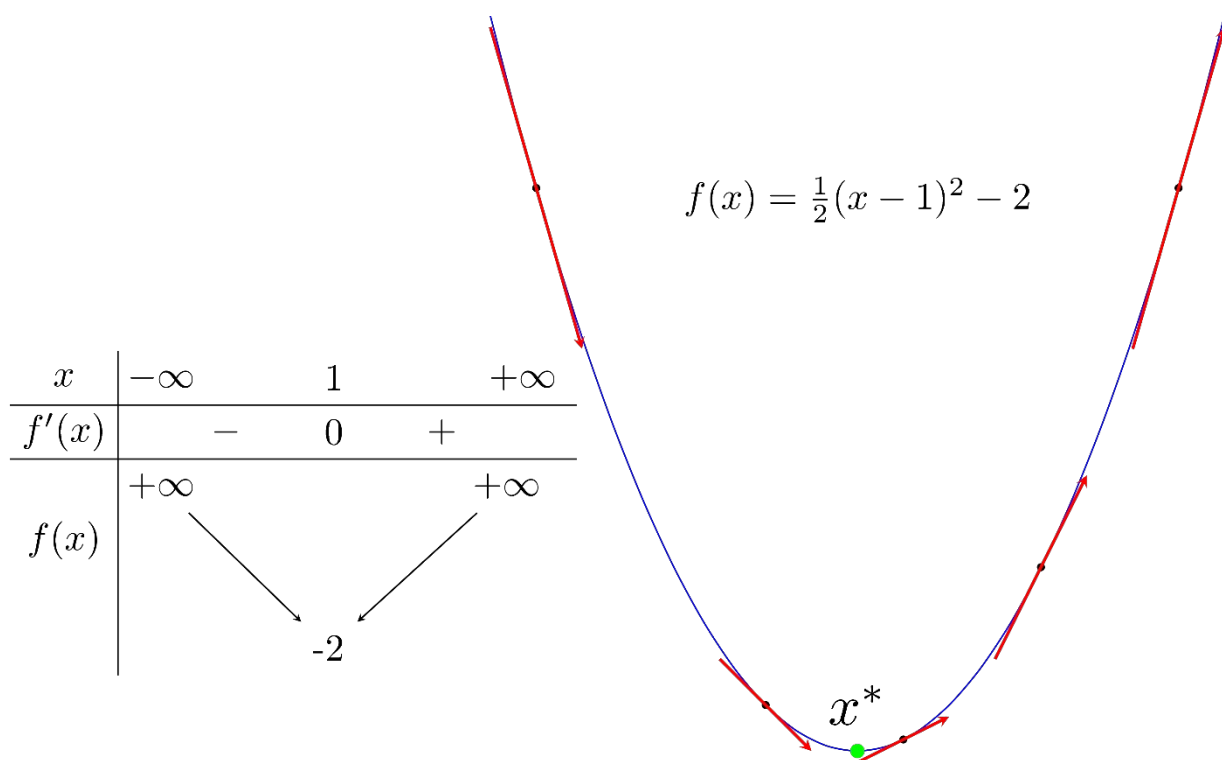
Mỗi lần duyệt qua tất cả các vector K trên toàn bộ dữ liệu được gọi là một epoch [3].

1.1.3. Thuật toán Gradient Descent

Trong toán tối ưu, chúng ta thường xuyên phải tìm các giá trị nhỏ nhất của một hàm số. Việc tìm giá trị nhỏ nhất của hàm số phức tạp là không khả thi, nên ta thường tìm các điểm cực tiểu cục bộ (local minimum), và coi đó là một nghiệm cần tìm của bài toán. Các điểm cực tiểu cục bộ là nghiệm của phương trình đạo hàm bằng không. Tuy nhiên, trong nhiều trường hợp, việc giải phương trình đạo hàm bằng không để tìm nghiệm chính xác cũng là không khả thi, nên cần có hướng giải quyết là tìm nghiệm gần đúng [3].

Hướng tiếp cận phổ biến nhất để giải quyết các bài toán tối ưu là xuất phát từ một điểm được coi là gần với nghiệm của bài toán, sau đó dùng một phép toán lặp để tiến dần đến điểm đạo hàm bằng không [3].

Xét hàm số một biến $f: R \rightarrow R$



Hình 1.3 Khảo sát sự biến thiên của một đa thức bậc 2 một biến [3]

Trên Hình 1.3 là sự biến thiên của một đa thức bậc 2 một biến. Điểm cực tiểu cục bộ thực sự của $f(x)$ là $x^* = -2$. Ta dùng thuật toán gradient descent để tìm điểm gần với x^* nhất. Từ Hình 1.3 ta có quan sát sau đây.

Gọi x_t là một điểm trên đồ thị có được sau t vòng lặp. Nếu đạo hàm của hàm số tại x_t là dương ($\frac{\partial f}{\partial x_t} > 0$) thì x_t nằm về bên phải so với x^* , và ngược lại nếu đạo hàm của hàm số tại x_t là âm ($\frac{\partial f}{\partial x_t} < 0$) thì x_t nằm bên trái so với x^* . Để điểm tiếp theo x_{t+1} gần với x^* hơn, ta cần di chuyển x_t ngược dấu với đạo hàm như công thức (1.19) [3].

$$x_{t+1} = x_t - \eta \frac{\partial f}{\partial x_t}, \quad (1.19)$$

trong đó, η là một số dương gọi là tốc độ học. Dấu trừ thể hiện việc chúng ta phải đi ngược chiều với đạo hàm. Tốc độ hội tụ của gradient descent phụ thuộc vào điểm xuất phát và tốc độ học. Các quan sát đơn giản phía trên, mặc dù không phải đúng trong tất cả các trường hợp, nhưng là nền tảng cho rất nhiều phương pháp tối ưu [3].

Gradient descent cho hàm nhiều biến cũng tương tự như cho hàm một biến, cũng bắt đầu từ một điểm dự đoán, mỗi vòng lặp ta đi ngược dấu với đạo hàm. Khi tính đạo hàm riêng của hàm số theo một biến thì các biến khác được coi là hằng số.

1.2. Giới thiệu về FPGA

1.2.1. Khái niệm FPGA

Field programmable gate array (viết tắt là FPGA) là mạch tích hợp bao gồm các khối có thể cấu hình và lập trình được. Ngoài ra, liên kết giữa các khối đó cũng có thể cấu hình và lập trình được. Kỹ sư có thể cấu hình các thiết bị FPGA để thực hiện nhiều tác vụ lớn lao [4].

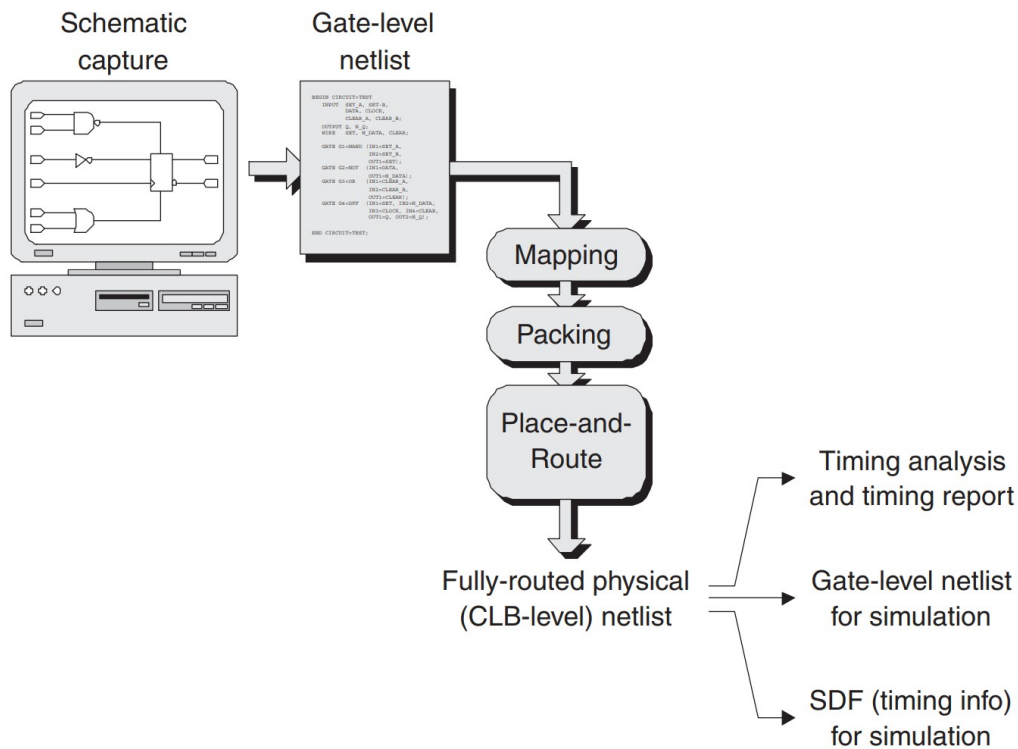
Phụ thuộc vào cách mà chúng được triển khai, một vài thiết bị FPGA có thể chỉ lập trình được một lần, một số khác có thể lập trình được nhiều lần. Thiết bị chỉ lập trình được một lần gọi là OTP (one-time programmable) [4].

Cái tên field programmable trong FPGA ám chỉ việc lập trình tại hiện trường để sử dụng, trái ngược với việc thiết bị được nối dây sẵn bên trong bởi nhà sản xuất để thực hiện chức năng nào đó. Điều đó cho phép các thiết bị FPGA có thể được lập trình trong phòng nghiên cứu, và có thể thay đổi được chức năng khi vẫn nằm trong hệ thống điện tử vốn đã được triển khai ngoài thực tế. Khi một thiết bị có khả năng lập trình mà vẫn giữ nguyên vị trí của nó trong hệ thống cao cấp hơn, nó được coi là ISP (in-system programmable) [4].

FPGA cấu tạo bởi các khối logic có thể lập trình được, gọi là CLB (configurable logic block), mỗi khối này được tạo nên bởi một số các LUT (look up table) và các thanh ghi (registers) [4].

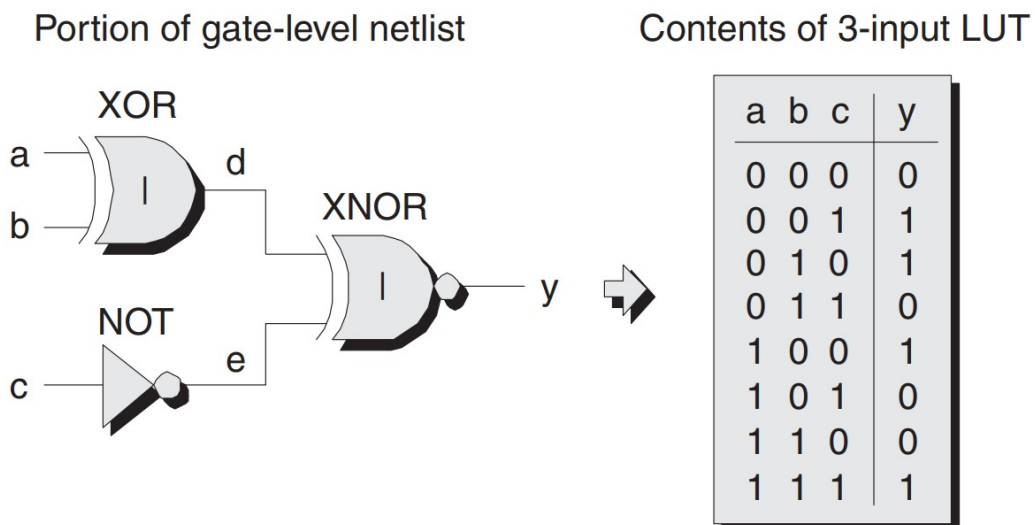
1.2.2. Schematic-based flow

Vào khoảng những năm 1960, người ta thiết kế mạch điện tử theo schematic-based flow, tức là luồng thiết kế dựa trên sơ đồ nguyên lý. Mạch điện tử được vẽ thủ công bằng giấy bút, mỗi đội thiết kế thường có một kỹ sư rất giỏi việc đơn giản hóa và tối ưu hóa mạch. Không chỉ có vậy, việc kiểm chứng timing có đạt yêu cầu kỹ thuật hay không cũng phải được tính toán thủ công. Vì thiết kế theo cách này rất mất thời gian nên nhiều công ty và các trường đại học đã tìm cách khác hiệu quả hơn, ví dụ như trong việc mô phỏng, vào khoảng những năm 1970, người ta tạo ra một công cụ mô phỏng thay cho việc mô phỏng thủ công. Vì công cụ này vẫn còn sơ khai nên các kỹ sư phải mô tả mạch dưới dạng một file text đại diện cho mạch ở mức độ cổng logic, gọi là gate-level netlist. Năm 1984 là năm các thiết bị FPGA đầu tiên được sản xuất, và chúng được thiết kế theo schematic-based flow với các bước được trình bày trên Hình 1.4 [4].



Hình 1.4 Thiết kế FPGA theo schematic-based flow [4]

Các kỹ sư sử dụng phần mềm để vẽ schematic, từ đó tạo ra gate-level netlist. Mapping là bước ánh xạ các thực thể trong netlist với các LUT sẵn có trong thiết bị. Mỗi LUT có thể dùng để đại diện cho nhiều cổng logic nên có thể map nhiều cổng logic nối với nhau trong netlist với một LUT, ví dụ như trên Hình 1.5, ánh xạ một nhóm các cổng logic vào một LUT có 3 đầu vào.



Hình 1.5 Mapping các cổng logic vào LUT [4]

Việc mapping là một vấn đề phức tạp bởi có nhiều cách để chia thiết kế ra thành nhiều phần nhỏ. Bước tiếp theo là packing, nghĩa là việc các LUT và thanh ghi được gói vào CLB. Place-and-route là việc đặt các CLB vào các vị trí hợp lý [4].

Sau đó, để kiểm tra các kết nối trong mạch có thỏa mãn các yêu cầu về timing hay không, ta sử dụng phương pháp static timing analysis để phân tích độ trễ của các đường trong mạch, kết quả là các file như sau:

Timing analysis and timing report là file báo cáo về setup time, holdtime, critical path và một vài tham số khác. Đối với thanh ghi, giả sử dữ liệu đồng bộ theo sườn lên của clock, setup time là khoảng thời gian tối thiểu đầu vào dữ liệu phải không đổi đến khi có sườn lên của clock, hold time là khoảng thời gian tối thiểu đầu vào dữ liệu phải không đổi từ khi có sườn lên của clock. Critical path là đường từ một input đến một output có delay lớn nhất.

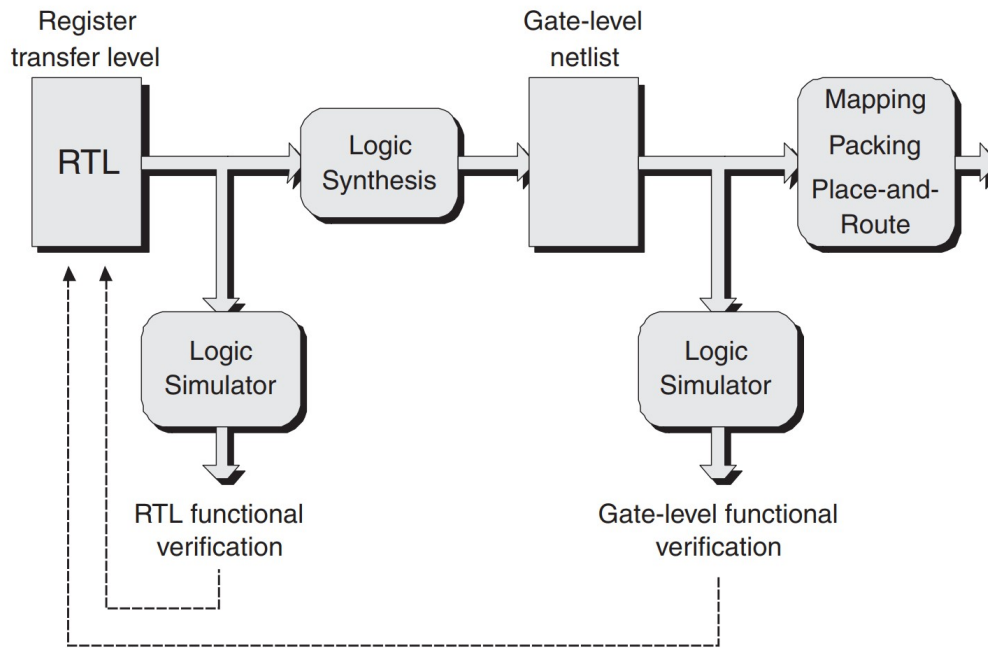
Gate-level netlist for simulation là file mô tả mạch ở mức độ các cell để mô phỏng. File gate-level netlist này khác với gate-level netlist tạo ở bước trước khi mapping.

SDF for simulation là file chứa thông tin về timing của từng cell trong gate-level netlist for simulation.

Ngày nay, toàn bộ các bước trên (schematic capture, gate-level netlist, mapping, packing, place-and-route) cùng với việc phân tích timing được các nhà sản xuất thiết bị FPGA làm sẵn kèm với tool của họ [4].

1.2.3. HDL-based flow

Đến khoảng cuối những năm 1980, các thiết kế không ngừng phát triển về kích thước và độ phức tạp, khiến cho việc thiết kế theo schematic-based flow không đáp ứng đủ nhu cầu bởi việc tưởng tượng, debug, hiệu, bảo trì,... một mạch ở cấp độ gồm toàn những cổng logic cơ bản là một điều rất khó khăn. Do đó người ta phát minh ra một thứ có thể mô tả mạch ở mức độ trừu tượng hơn, gọi là hardware discription language (HDL), nghĩa là ngôn ngữ mô tả phần cứng [4].



Hình 1.6 Thiết kế FPGA theo HDL-based flow [4]

Hình 1.6 minh họa cho thiết kế FPGA theo HDL-based flow, tức là thiết kế theo luồng dựa trên ngôn ngữ mô tả phần cứng. Register transfer level (RTL) trên hình thể hiện việc dùng HDL để mô tả mạch ở mức độ thanh ghi trừu tượng. Logic simulator là việc mô phỏng để kiểm chứng chức năng của mạch. Logic synthesis là việc biến đổi mạch thể hiện ở dạng RTL trước đó thành các cổng logic cơ bản trong gate-level netlist. Các bước mapping, packing, place-and-route vẫn giống như schematic-based flow. [4]

Hiện nay, các tool của Quartus và Xilinx có thể tự động làm các bước logic synthesis, mapping, packing, place-and-route mà không cần sự can thiệp của con người.

1.3.

1.4. Giới thiệu về SystemVerilog

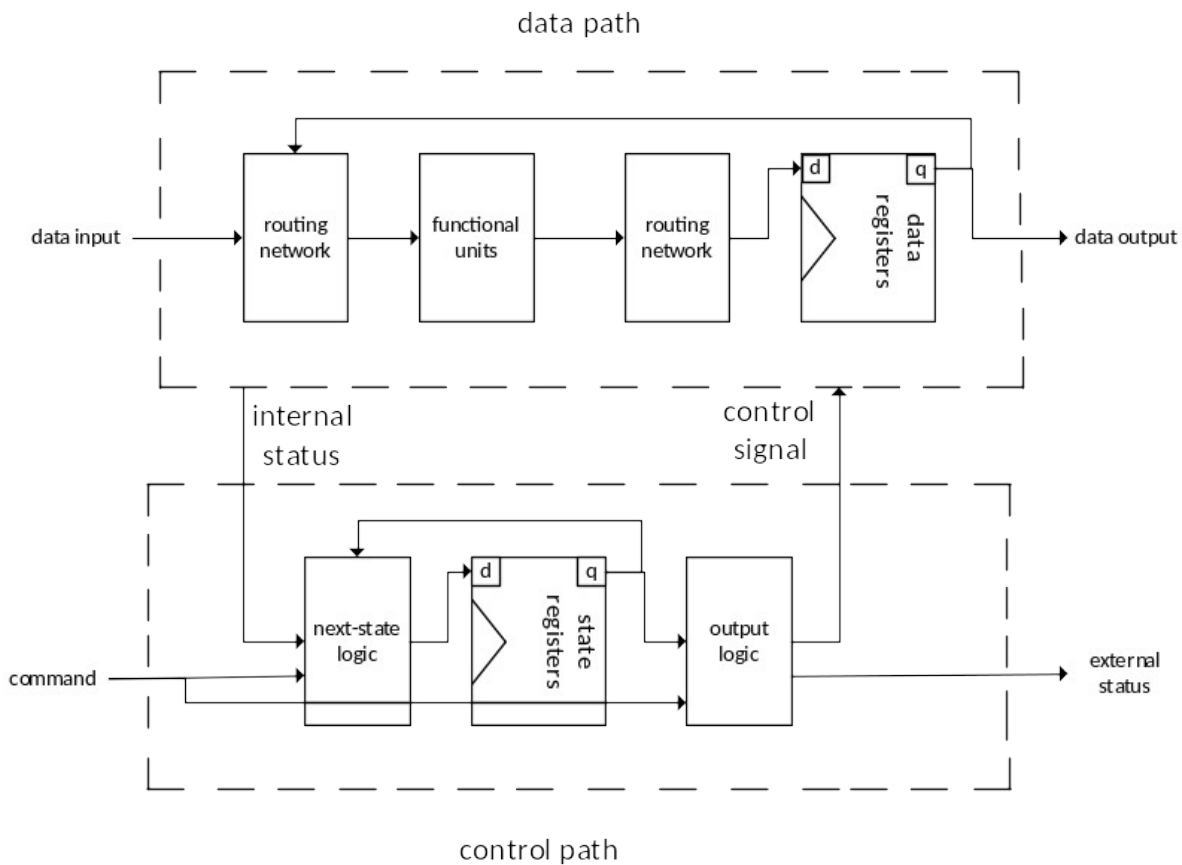
SystemVerilog là một loại HDL được mở rộng từ Verilog-2005, bổ sung thêm nhiều đặc điểm của ngôn ngữ C, C++, Superlog, Một số đặc điểm hỗ trợ việc viết các mô hình phần cứng có thể synthesis được là [5]:

- Có kiểu dữ liệu interface để đóng gói sự giao tiếp và kiểm tra giao thức giữa các khối trong thiết kế.
- Có các kiểu dữ liệu giống như C, ví dụ như int.
- Có kiểu dữ liệu người dùng tự định nghĩa là typedef.
- Có enum.
- Có thể ép kiểu dữ liệu.
- Có struct và union.
- Có kiểu dữ liệu là package được chia sẻ bởi nhiều khối thiết kế.
- Có lựa chọn ưu tiên hoặc duy nhất trong case.
- Có thể truyền tham chiếu trong task, function hoặc module.

Tóm lại, SystemVerilog hợp nhất nhiều ưu điểm của các ngôn ngữ mô tả và kiểm chứng phần cứng ở dạng mở rộng của Verilog. Sự mở rộng này làm cho khả năng xây dựng và mô phỏng RTL rất mạnh mẽ. Bởi vậy em đã chọn SystemVerilog là ngôn ngữ để thiết kế sản phẩm trong đồ án này.

1.5. Giới thiệu về mô hình FSM/D

Một FSM/D là sự kết hợp của FSM với mạch tuần tự thông thường, mạch tuần tự này được gọi là data path, với chữ D viết tắt trong FSM/D ám chỉ data path. Còn FSM, thường được coi là control path, khảo sát các yêu cầu bên ngoài và trạng thái của mạch để đưa ra lệnh chỉ định hoạt động của mạch tuần tự. FSM/D được sử dụng để triển khai các hệ thống được mô tả ở mức RTL, là mô hình mà dữ liệu được vận chuyển giữa các thanh ghi. Sơ đồ khối của FSM/D được mô tả trên Hình 1.7 [6].



Hình 1.7 Sơ đồ khối của FSMD [6]

Sơ đồ khối Hình 1.7 thể hiện khái niệm của FSMD, được chia thành hai phần là data path và control path. Phần data path làm nhiệm vụ vận chuyển dữ liệu giữa các thanh ghi và bao gồm các khối con:

- Data registers: chứa các kết quả trung gian trong việc tính toán.
- Functional units: thực hiện các hàm để tính toán với dữ liệu.
- Routing network: phân luồng dữ liệu giữa data registers và functional unit.

Control path tạo ra control signal để điều khiển data path, và data path tạo ra internal status để phản hồi lại control path. Control path dựa vào command từ bên ngoài và internal status để chuyển trạng thái và đưa ra control signal. Control path là một FSM, có state register là trạng thái hiện tại, next-state logic là trạng thái tiếp theo, khối output logic dựa vào trạng thái hiện tại và command để đưa ra control signal và external status, với external status là tín hiệu thể hiện trạng thái hiện tại của mạch [6].

Sản phẩm của đồ án này được thiết kế theo mô hình FSMD của Pong P.Chu với cơ sở lý thuyết đã được trình bày như ở trên đây, chi tiết về thiết kế được trình bày trong phần sau.

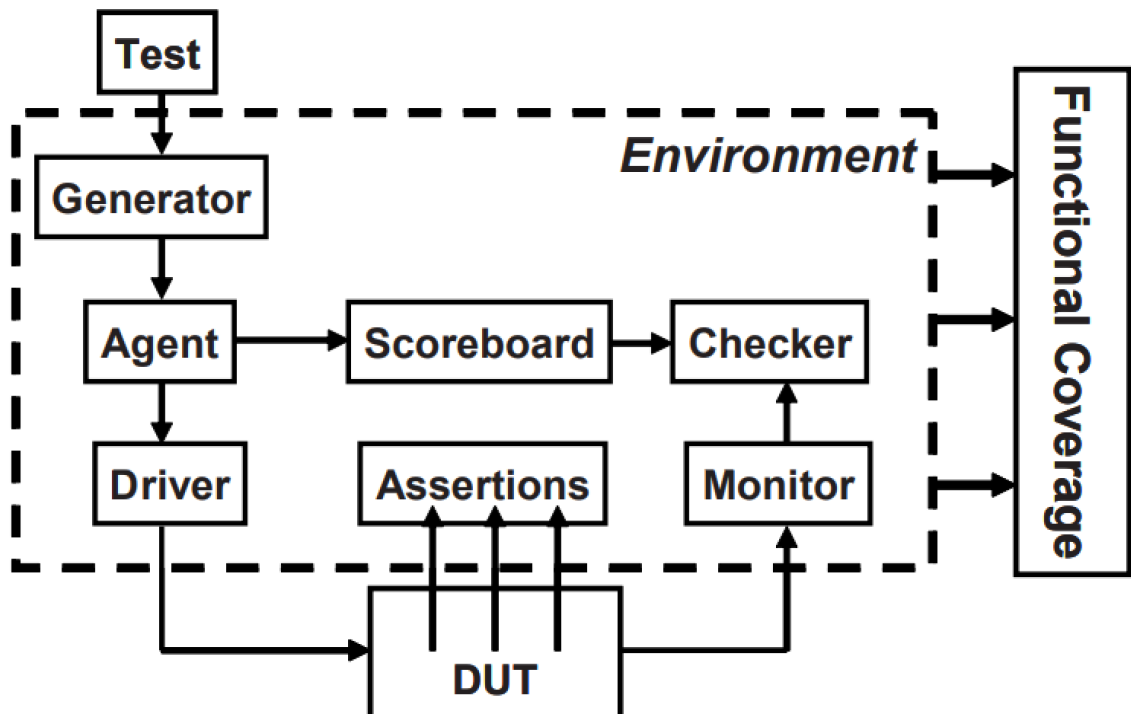
1.6. Giới thiệu về kiểm chứng thiết kế

Mục đích của kiểm chứng thiết kế (verification) là đảm bảo thiết bị hoàn thiện thành công tất cả các nhiệm vụ, tức là thiết kế phải thể hiện đúng yêu cầu kỹ thuật (specification). Một kỹ sư kiểm chứng cần phải đọc yêu cầu kỹ thuật, tạo kế hoạch kiểm chứng (verification plan), và bám vào nó để xây dựng testbench xem code RTL có triển khai đúng các đặc điểm của yêu cầu kỹ thuật không. Kế hoạch kiểm chứng có quan hệ chặt chẽ với yêu cầu kỹ thuật và bao gồm mô tả về các đặc điểm nào cần phải kiểm tra và kiểm tra bằng phương thức nào. Testbench có mục đích là xem xét sự đúng đắn của thiết kế trong khi kiểm tra, gọi là DUT (design under test) [7]. Testbench được tạo thành bởi các bước:

- Tạo ra stimulus
- Đưa stimulus vào DUT
- Thu nhận phản hồi từ DUT
- Kiểm tra sự đúng đắn
- Đo đặc tính độ so với các mục tiêu đã đề ra

Trong các bước trên, có một vài bước có thể được testbench tự động tạo ra, một số bước có thể người viết testbench phải tự tùy chỉnh và xem xét.

Testbench có thể được xây dựng theo kiến trúc phân lớp như Hình 1.8.



Hình 1.8 Mô hình testbench đủ các lớp [7]

Trên Hình 1.8 là mô hình testbench với đủ các lớp. Thực tế một testbench có thể không cần phải có đủ các lớp như trên, hoặc có thể có thêm một số lớp khác, điều đó phụ thuộc vào DUT cụ thể như thế nào, chức năng của mỗi lớp trong hình trên được mô tả ngắn gọn như sau, với chiều mũi tên ám chỉ chiều của luồng dữ liệu [7]:

- Lớp Functional Coverage (bao phủ chức năng) có nhiệm vụ đo đặc tiến độ, tức là đã kiểm chứng được bao nhiêu đặc điểm được liệt kê trong kế hoạch kiểm chứng. Nhưng ngay cả khi độ bao phủ chức năng đạt 100% thì thiết kế vẫn có thể tiềm ẩn lỗi bởi kế hoạch kiểm chứng có thể không bao quát được hết các trường hợp phát sinh lỗi, vì vậy cần liên tục cập nhật kế hoạch kiểm chứng mới. Một kế hoạch kiểm chứng chỉ kết thúc khi functional coverage đạt 100%.
- Lớp Generator tạo ra các stimulus ở cấp độ trừu tượng.
- Lớp Agent biến đổi các stimulus thành các lệnh ít trừu tượng hơn.
- Lớp Driver nhận lệnh từ Agent và biến đổi lệnh thành các tín hiệu logic để gán vào các input của DUT.
- Lớp Monitor thu nhận output của DUT.
- Lớp Checker biến đổi giá trị của output mà monitor thu được thành các giá trị ở mức trừu tượng hơn.
- Lớp Assertion kiểm tra tính hợp lệ về thời gian đối với các giao thức của DUT, xem các tín hiệu trong DUT thay đổi có thỏa mãn quan hệ về thời gian hay không.
- Lớp Scoreboard so sánh đầu ra với giá trị kỳ vọng.
- Lớp Environment là lớp bao gồm Generator, Agent, Driver, Monitor, Assertion, Checker và Scoreboard.
- Lớp Test điều khiển hoạt động của Environment.
- Khối DUT là thiết kế đang cần kiểm chứng.

1.7. Kết luận chương

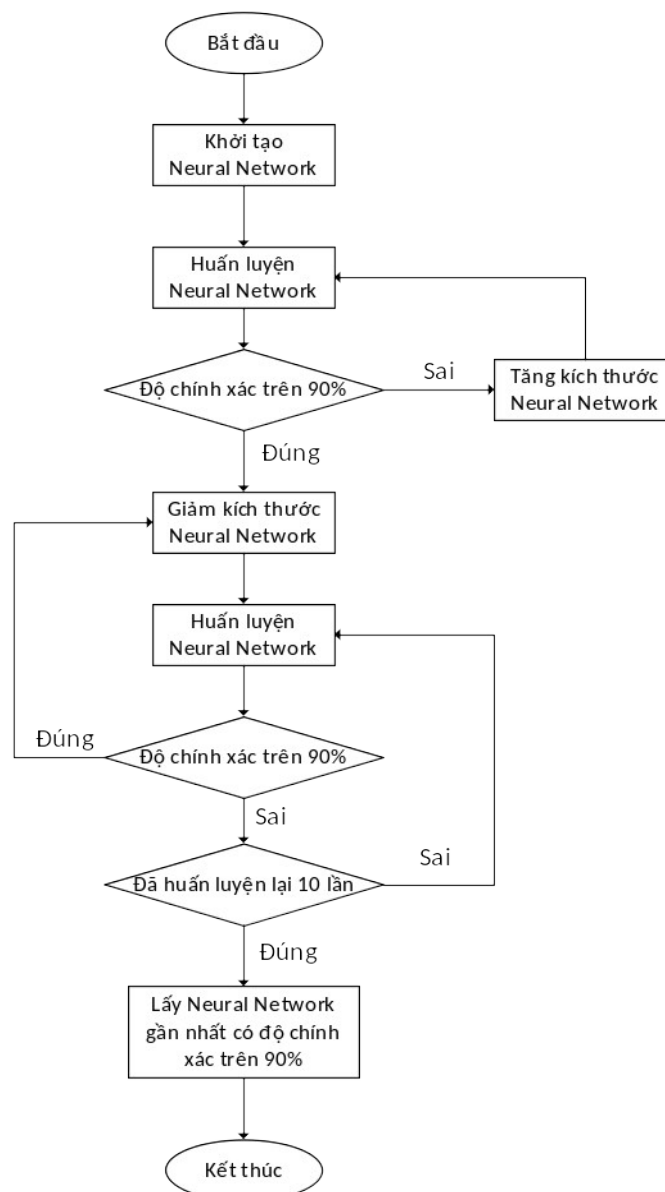
Chương này đã giới thiệu những kiến thức tổng quát về Neural Network, FPGA, ngôn ngữ SystemVerilog, mô hình FSMĐ và kiểm chứng thiết kế, làm tiền đề cho việc thiết kế và kiểm chứng trong các chương sau.

CHƯƠNG 2. THIẾT KẾ NEURAL NETWORK VỚI MATLAB

Việc thiết kế Neural Network trong chương này được triển khai dựa trên cơ sở lý thuyết đã trình bày trong phần 1.1, với công cụ làm việc là Matlab. Mục tiêu là tìm ra được các tham số weight và bias sao cho mạng đạt độ chính xác trên 90% với tập kiểm tra MNIST, đồng thời kích thước của mạng cũng là nhỏ nhất có thể mà vẫn đảm bảo độ chính xác đó. Kết quả quan trọng đã thu được là các tham số weight và bias thỏa mãn yêu cầu trên, và sẽ được dùng để triển khai trên FPGA.

2.1. Thuật toán thiết kế

Việc thiết kế được thực hiện theo lưu đồ thuật toán như Hình 2.1.



Hình 2.9 Lưu đồ thuật toán thiết kế trên Matlab

Sau khi tìm được mạng cho độ chính xác trên 90%, ta tiến hành giảm kích thước của mạng xuống rồi huấn luyện lại để xem độ chính xác có còn đạt 90% nữa hay không. Khi kích thước của mạng giảm xuống một mức độ nhất định, ta huấn luyện tới 10 lần mà vẫn không có được độ chính xác trên 90% thì ta dừng lại và chọn mạng gần nhất có độ chính xác trên 90% làm kết quả. Như vậy, ta cần huấn luyện nhiều lần để tìm ra mạng có kích thước nhỏ nhất có thể cho kết quả độ chính xác trên 90%.

Ban đầu mạng được khởi tạo với 100 nút tại lớp ẩn, mỗi lần cần giảm kích thước mạng thì ta giảm đi 5 nút tại lớp ẩn. Kết quả cuối cùng thu được là mạng với 15 nút tại lớp ẩn.

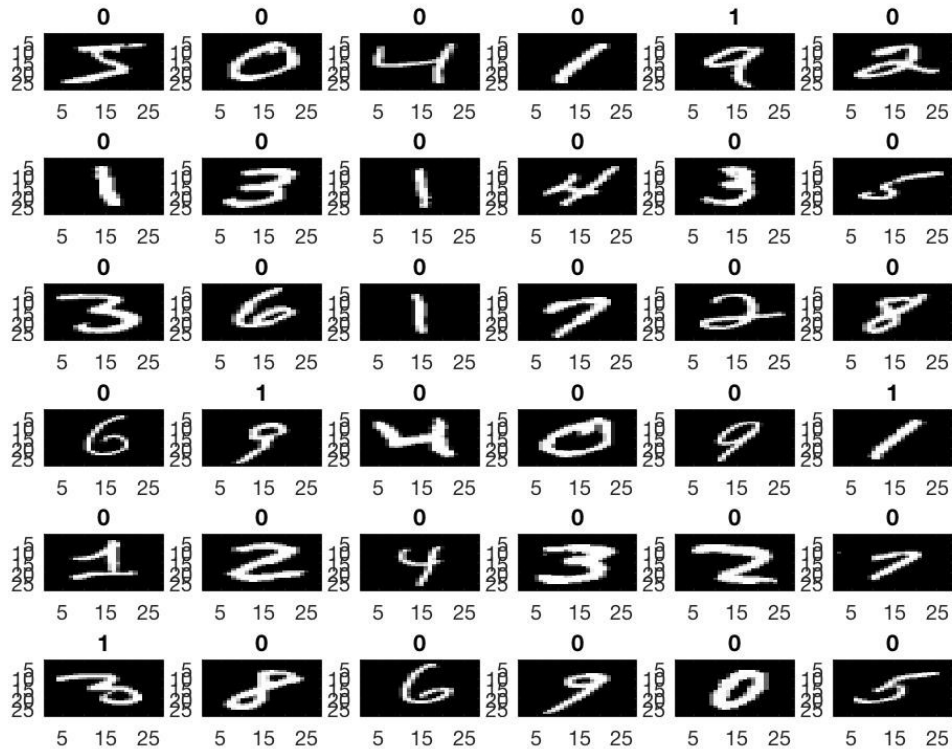
Bước Huấn luyện Neural Network được trình bày trong phần 2.2. Bước kiểm tra cho Độ chính xác trên 90% được trình bày trong phần 2.3.

2.2. Huấn luyện trên Matlab

2.2.1. Chuẩn bị cơ sở dữ liệu

Cơ sở dữ liệu là cơ sở dữ liệu chữ số viết tay MNIST được tải về từ Internet. Cơ sở dữ liệu này được chia thành hai tập, tập huấn luyện bao gồm 60000 mẫu và tập kiểm tra bao gồm 10000 mẫu. Mỗi bức ảnh chứa một chữ số được chuẩn hóa kích thước 28x28 điểm ảnh, là ảnh xám và mỗi chữ số đều được đặt tại trung tâm của mỗi ảnh [7].

Cơ sở dữ liệu MNIST được cấu tạo từ Special Database 3 (SD-3) và Special Database 1 (SD-1) của tổ chức NIST, bao gồm các ảnh nhị phân của chữ số viết tay. Ban đầu, NIST đã thiết kế SD-3 là tập huấn luyện và SD-1 là tập kiểm tra. Tuy nhiên, SD-3 lại được nhận dạng rõ ràng và chính xác hơn SD-1. Lý do là bởi SD-3 được thu thập từ các nhân viên công ty, còn SD-1 được thu thập từ các học sinh phổ thông. Do đó NIST đã xây dựng cơ sở dữ liệu này là sự pha trộn của hai cơ sở dữ liệu trên. Tập huấn luyện gồm 30000 mẫu từ SD-3 và 30000 mẫu từ SD-1. Tập kiểm tra gồm 5000 mẫu từ SD-3 và 5000 mẫu từ SD-1. Tập huấn luyện có tổng cộng 60000 mẫu được thu thập từ khoảng 250 người viết. Việc chia tập huấn luyện và tập kiểm tra đã được đảm bảo rằng không có người nào vừa viết tập huấn luyện vừa viết tập kiểm tra [7].



Hình 2.10 Cơ sở dữ liệu MNIST

Hình 2.2 là 36 chữ số đầu tiên của tập huấn luyện trong cơ sở dữ liệu chữ số viết tay MNIST. Ta sử dụng 60000 ảnh trong tập huấn luyện MNIST làm tập huấn luyện.

2.2.2. Tiền xử lý huấn luyện

Mục tiêu của tiền xử lý huấn luyện là biến đổi 60000 bức ảnh trong tập huấn luyện MNIST thành đầu vào phù hợp với Neural Network. Quá trình tiền xử lý gồm các bước sau:

Bước 1: Xếp ảnh từ ma trận 28×28 thành một vector 784×1 .

Bước 2: Chia mỗi phần tử trong vector cho 255.

Bước 3: Mapminmax để chuẩn hóa.

Bước 4: Loại bỏ các hàng hằng số.

Ta cần có Bước 1 là để đưa ảnh từ một ma trận vuông thành một vector, phù hợp với đầu vào tại lớp đầu vào của Neural Network.

Vì ảnh là ảnh xám (grayscale) 8 bit nên giá trị các điểm ảnh ban đầu là các số nguyên không nhỏ hơn 0 và không vượt quá 255. Sau khi thực hiện Bước 2 thì giá trị các điểm ảnh là các số thực không nhỏ hơn 0 và không vượt quá 1. Thực nghiệm cho

thấy nếu không chuẩn hóa thì giá trị các tham số weight và bias sau huấn luyện sẽ rất nhỏ, gây khó khăn cho việc biểu diễn dữ liệu trên FPGA.

Tại Bước 3, mapminmax là việc biến đổi một vector $X=[x_1, x_2, \dots, x_N]$ thành vector $Y=[y_1, y_2, \dots, y_N]$ theo công thức sau:

$$\frac{y_i - y_{\min}}{y_{\max} - y_{\min}} = \frac{x_i - x_{\min}}{x_{\max} - x_{\min}} \text{ với } i=1, 2, \dots, N \quad (2.1)$$

Trong công thức (1.20) ta có $x_{\max}$ là phần tử lớn nhất của X , $x_{\min}$ là phần tử nhỏ nhất của X . Chọn $y_{\max}=1$ và $y_{\min}=-1$ thì các phần tử của Y sẽ nằm trong khoảng từ -1 đến 1. Như vậy sau Bước 3 thì 60000 vector ảnh huấn luyện đều được chuẩn hóa trong khoảng từ -1 đến 1.

Tại Bước 4, ta ghép 60000 vector lại thành một ma trận cỡ 784x60000 rồi tìm kiếm trong các hàng của ma trận đó có hàng nào mà giá trị không thay đổi thì bỏ hàng đó đi. Các điểm nằm trên cùng một hàng của ma trận chính là các điểm có cùng tọa độ trên các ảnh, nếu chúng giống nhau thì các điểm đó không có ý nghĩa phân biệt các bức ảnh. Việc này là để loại bỏ bớt dữ liệu không có tác dụng cho việc nhận dạng, làm cho neural network giảm kích thước. Sau khi thực hiện loại bỏ các hàng hằng số thì ma trận giảm kích thước xuống còn 717x60000, hay nói cách khác là tuy ảnh có 784 điểm ảnh nhưng chỉ có 717 điểm ảnh có tác dụng nhận dạng.

Kết quả khi thực hiện 4 bước trên là:

- Ma trận I cỡ 717x60000, mỗi cột là một đầu vào của neural network.
- Vector C lưu số thứ tự các hàng bị loại bỏ tại bước loại bỏ các hàng hằng số.

Ma trận I sẽ được dùng cho việc huấn luyện Neural Network, còn vector C sẽ được dùng cho quá trình tiền xử lý ảnh khi kiểm tra.

2.2.3. Huấn luyện Neural Network

Mục tiêu của huấn luyện Neural Network là để thu được các tham số weight và bias tốt nhất có thể, sau đó sử dụng các tham số này cho việc nhận dạng. Việc huấn luyện đã được thực hiện trên Matlab, có thể chia nhỏ thành các bước sau:

Bước 1: Tạo đầu vào và kỳ vọng cho Neural Network.

Bước 2: Cấu hình Neural Network.

Bước 3: Phân chia đầu vào.

Bước 4: Huấn luyện.

Tại Bước 1, đầu vào của Neural Network chính là các cột của ma trận I cỡ 717x60000 đã thu được tại bước tiền xử lý huấn luyện. Mỗi cột của ma trận I đại diện

cho một ảnh trong tập huấn luyện đã biết trước là chữ số nào, từ đó ta có giá trị của giám sát viên. Dựa trên chữ số trên ảnh, ta mã hóa giám sát viên theo kiểu one-hot như trong Bảng 2.1. Như vậy, giám sát viên của 60000 đầu vào là 60000 vector cỡ 10x1 mã hóa theo kiểu one-hot.

Bảng 2.1 Mã hóa giám sát viên theo chữ số

Chữ số	Giám sát viên [10:1]
1	0000000001
2	0000000010
3	0000000100
4	0000001000
5	0000010000
6	0000100000
7	0001000000
8	0010000000
9	0100000000
0	1000000000

Tại Bước 2, ta cấu hình Neural Network theo mô hình 3 lớp đã trình bày trong phần 1.2. Tuy nhiên, kích thước của các lớp, các weight và bias, hàm kích hoạt của các lớp được tùy biến để phù hợp với mục tiêu là nhận dạng chữ số viết tay.

Lớp đầu vào là vector có 717 phần tử:

$$K = \begin{bmatrix} K_1 \\ \dots \\ K_{717} \end{bmatrix} \text{ có kích thước } 717 \times 1.$$

Lớp ẩn có đầu vào và đầu ra là các vector có 15 phần tử:

$$Z_2 = \begin{bmatrix} z_1^2 \\ \dots \\ z_{15}^2 \end{bmatrix} \text{ có kích thước } 15 \times 1,$$

$$A2 = \begin{bmatrix} a_1^2 \\ \dots \\ a_{15}^2 \end{bmatrix} \text{ có kích thước } 15 \times 1.$$

Hàm kích hoạt tại lớp ẩn được sử dụng là hàm tansig có công thức như (2.2):

$$a_i^2 = \text{tansig}(z_i^2) = \frac{2}{1 + e^{-2z_i^2}} - 1 \text{ với } i = 1, 2, \dots, 15 \quad (2.2)$$

Lớp đầu ra có đầu vào và đầu ra là các vector có 10 phần tử:

$$Z3 = \begin{bmatrix} z_1^3 \\ \dots \\ z_{10}^3 \end{bmatrix} \text{ có kích thước } 10 \times 1,$$

$$A3 = \begin{bmatrix} a_1^3 \\ \dots \\ a_{10}^3 \end{bmatrix} \text{ có kích thước } 10 \times 1.$$

Hàm kích hoạt tại lớp đầu ra là hàm softmax có công thức như (2.3):

$$a_i^3 = \text{softmax}(z_i^3) = \frac{e^{z_i^3}}{\sum (e^{z_i^3})} \quad (2.3)$$

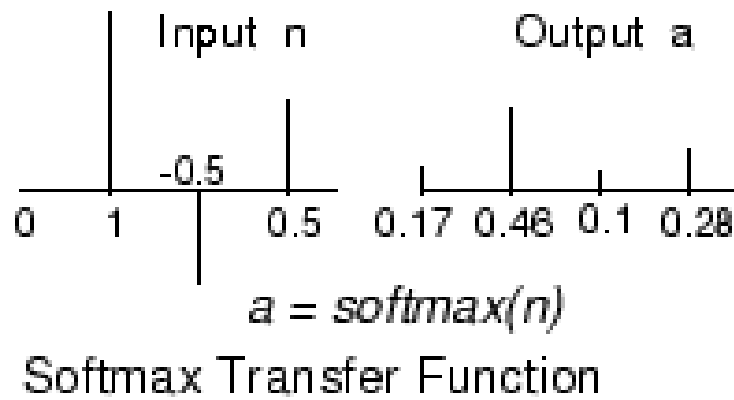
Hình 2.3 là minh họa cho hàm $a = \text{softmax}(n)$. Cách tính như sau:

$$a_1 = \frac{e^0}{e^0 + e^1 + e^{-0.5} + e^{0.5}} = 0.17 \quad (2.4)$$

$$a_1 = \frac{e^1}{e^0 + e^1 + e^{-0.5} + e^{0.5}} = 0.46 \quad (2.5)$$

$$a_1 = \frac{e^{-0.5}}{e^0 + e^1 + e^{-0.5} + e^{0.5}} = 0.10 \quad (2.6)$$

$$a_1 = \frac{e^{0.5}}{e^0 + e^1 + e^{-0.5} + e^{0.5}} = 0.28 \quad (2.7)$$



Hình 2.11 Hàm softmax với vector 4 phần tử

Các weight và bias có kích thước như sau:

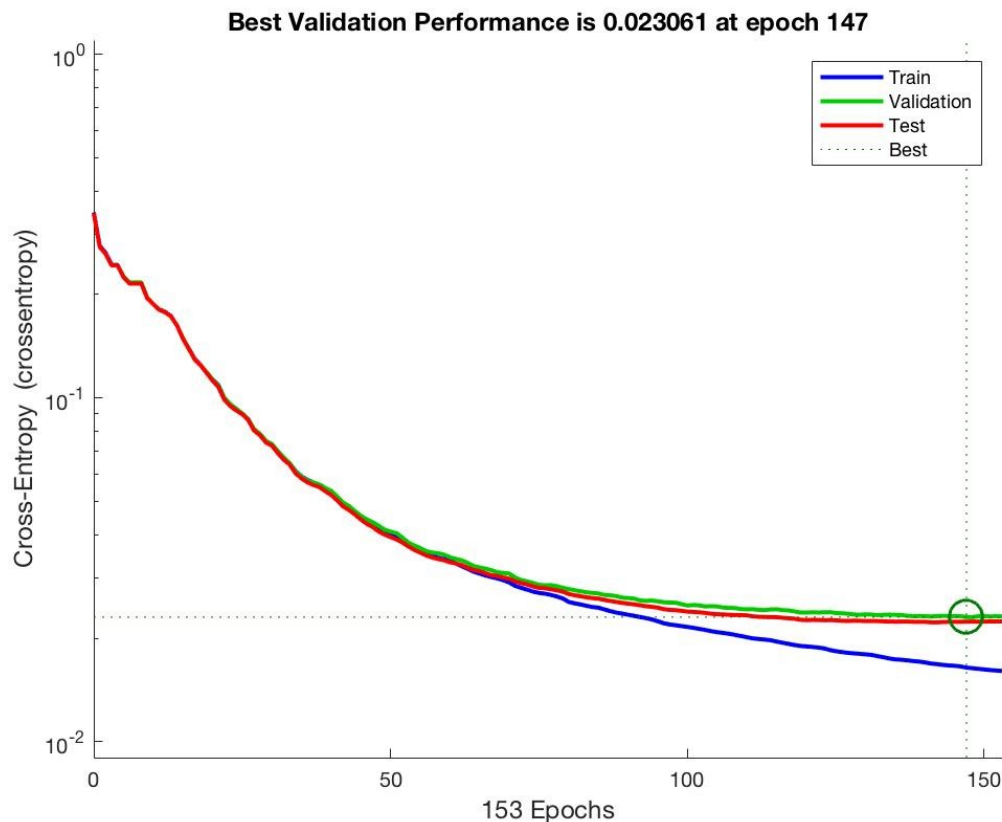
$$W2 = \begin{bmatrix} w_{1,1}^2 & \dots & w_{717,1}^2 \\ \dots & \dots & \dots \\ w_{1,15}^2 & \dots & w_{717,15}^2 \end{bmatrix} \text{ có kích thước } 15 \times 717,$$

$$B2 = \begin{bmatrix} b_1^2 \\ \dots \\ b_{15}^2 \end{bmatrix} \text{ có kích thước } 15 \times 1,$$

$$W3 = \begin{bmatrix} w_{1,1}^3 & \dots & w_{15,1}^3 \\ \dots & \dots & \dots \\ w_{1,10}^3 & \dots & w_{15,10}^3 \end{bmatrix} \text{ có kích thước } 10 \times 15,$$

$$B3 = \begin{bmatrix} b_1^3 \\ \dots \\ b_{10}^3 \end{bmatrix} \text{ có kích thước } 10 \times 1.$$

Tại Bước 3, ta cần phân chia số lượng 60000 vector đầu vào theo tỷ lệ 70% cho bộ huấn luyện (train set), 15% cho bộ xác nhận (validation set) và 15% cho bộ kiểm tra (test set). Ta huấn luyện với các mẫu trong bộ huấn luyện rồi kiểm tra sai lệch trên cả bộ huấn luyện, bộ xác nhận và bộ kiểm tra, quá trình sẽ dừng lại khi sai lệch trên bộ xác nhận và bộ kiểm tra giảm tới mức ổn định.



Hình 2.12 Hiệu suất huấn luyện

Tại Bước 4, việc huấn luyện được thực hiện theo cơ sở lý thuyết là thuật toán lan truyền ngược đã trình bày tại phần 1.1. Sau khi huấn luyện, ta có hiệu suất huấn luyện thể hiện trên Hình 2.4, với đường màu xanh lá cây là sai lệch của bộ xác nhận, đường màu đỏ là sai lệch của bộ kiểm tra và đường màu xanh lam là sai lệch của bộ huấn luyện. Ta thấy việc huấn luyện dừng lại sau 153 epoch, và bộ tham số tại epoch thứ 147 được coi là tốt nhất bởi vì từ epoch thứ 148 trở đi, tuy sai lệch của bộ huấn luyện có giảm xuống, nhưng sai lệch của bộ xác nhận và bộ kiểm tra vẫn không đổi, thì tức là mạng lúc này đã đạt được tính tổng quát. Nếu tiếp tục huấn luyện thì có thể mạng sẽ nhận dạng rất chính xác với bộ huấn luyện nhưng lại chưa chắc đã đúng với tập khác, không đảm bảo tính tổng quát, gọi là bị overfitting. Cơ sở của việc phân chia tập 60000 ảnh huấn luyện thành 3 tập con này là để tránh overfitting.

2.3. Kiểm tra trên Matlab

2.3.1. Chuẩn bị cơ sở dữ liệu

Như đã trình bày trong phần 2.2, cơ sở dữ liệu MNIST bao gồm 60000 ảnh huấn luyện và 10000 ảnh kiểm tra. Cơ sở dữ liệu sử dụng cho việc kiểm tra chính là 10000 ảnh đó. Như đã trình bày rằng, không có người nào vừa viết trong tập huấn luyện lại vừa viết trong tập kiểm tra, nên kết quả kiểm tra sẽ là khách quan và đáng tin cậy.

2.3.2. Tiền xử lý kiểm tra

Công việc cơ bản của kiểm tra là đưa các ảnh trong tập kiểm tra MNIST qua Neural Network rồi thống kê kết quả xem độ chính xác có đạt trên 90% hay không. Nhưng trước khi đưa ảnh vào mạng thì ta cần tiền xử lý để biến đổi ảnh về dạng phù hợp với đầu vào của mạng. Quá trình tiền xử lý cho ảnh kiểm tra cũng trải qua các bước như sau:

Bước 1: Xếp ảnh từ ma trận 28×28 thành một vector 784×1 .

Bước 2: Chia mỗi phần tử trong vector cho 255

Bước 4: Mapminmax để chuẩn hóa

Bước 5: Loại bỏ các hàng hằng số.

Với Bước 1, Bước 2 và Bước 3 ta làm giống như đối với tiền xử lý huấn luyện. Kết quả thu được là một vector T cỡ 784×10000 (thể hiện là có 10000 ảnh, mỗi ảnh có $28 \times 28 = 784$ điểm ảnh).

Tại Bước 4 có một chút khác biệt so với tiền xử lý huấn luyện. Sau Bước 4 Loại bỏ các hàng hằng số trong tiền xử lý huấn luyện, ta đã có một vector C lưu vị trí các hàng của ma trận I sẽ bị bỏ đi để tránh dư thừa dữ liệu. Tại bước này, ta áp dụng vector C đó để bỏ đi các hàng của T mà vị trí của nó có trong C . Sau 4 bước tiền xử lý ta thu được ma trận M cỡ 717×10000 với mỗi cột của ma trận đó đại diện cho một ảnh kiểm tra.

2.3.3. Kiểm tra Neural Network

Tại bước kiểm tra, ta sẽ truyền từng cột của ma trận M qua Neural Network đã được huấn luyện tại Chương 2. Việc đó được cụ thể hóa bằng các phép nhân ma trận và hàm kích hoạt như công thức (2.8), (2.9), (2.10), (2.11):

$$Z_2 = W_2 \cdot K + B_2 \quad (2.8)$$

$$A_2 = A = \text{tansig}(Z_2) \quad (2.9)$$

$$Z_3 = W_3 \cdot A_2 + B_3 \quad (2.10)$$

$$A_3 = \text{softmax}(Z_3) \quad (2.11)$$

trong đó K là vector đầu vào cỡ 717×1 . Các tham số đã có được do huấn luyện là gồm:

W_2 là ma trận chứa các weight tại lớp ẩn, cỡ 15×717

B_2 là ma trận chứa các bias tại lớp ẩn, cỡ 15×1

W_3 là ma trận chứa các weight tại lớp đầu ra, cỡ 10×15

B3 là ma trận chứa các bias tại lớp đầu ra, cỡ 10×1 .

Dễ thấy các công thức (2.8), (2.9), (2.10), (2.11) chính là các công thức (1.6), (1.7), (1.8), (1.9) trong phần nguyên lý hoạt động của Neural Network.

Đầu ra của Neural Network là kết quả của hàm softmax. Vì kết quả của hàm softmax không thể đạt tới giá trị 1 nên đầu ra khi kiểm tra không bao giờ bằng được với giám sát viên (mã hóa kiểu one-hot). Để đánh giá kết quả, ta làm tròn đầu ra của hàm softmax thành hàm “hardmax”, tức là phần tử lớn nhất trong 10 phần tử của vector sẽ được làm tròn thành 1 và 9 phần tử còn lại sẽ làm tròn về 0. Kết quả sau khi làm tròn mà trùng với giám sát viên thì ta coi đó là nhận dạng đúng.

Kết quả kiểm tra với 10000 ảnh trong tập kiểm tra MNIST được thống kê trong Bảng 2.2, với cột PASS là số lượng các mẫu được nhận dạng đúng, cột FAIL là số lượng các mẫu bị nhận dạng sai.

Bảng 2.2 Kết quả kiểm tra Neural Network trên Matlab

Chữ số	SUM	PASS	FAIL	% PASS
0	980	958	22	97.75
1	1135	1115	20	98.24
2	1032	966	66	93.60
3	1010	936	74	92.67
4	982	931	51	94.80
5	892	802	90	89.91
6	958	913	45	95.30
7	1028	968	64	94.16
8	974	899	75	92.30
9	1009	929	80	92.07

Độ chính xác trung bình lấy theo trung bình cộng của cột (% PASS) là 94.08%. Như vậy việc huấn luyện đã đạt yêu cầu là độ chính xác đạt trên 90% đối với tập kiểm tra MNIST. Ta sẽ triển khai Neural Network này trên FPGA với mục tiêu là độ chính xác trên FPGA cũng đạt trên 90%.

2.4. Kết luận chương

Neural Network thu được đạt độ chính xác trên 90% đối với tập MNIST là kết quả quan trọng nhất, vì ta lấy đó làm mục tiêu để xây dựng Neural Network trên FPGA. Ngoài ra, để việc triển khai trên FPGA khả thi thì kích thước của mạng cũng là yếu tố quan trọng. Trong quá trình tìm tòi nghiên cứu, mạng đã nhiều lần được thay đổi kích thước, huấn luyện lại nhiều lần mới tìm được kích thước nhỏ nhất có thể mà vẫn giữ được độ chính xác trên 90% đã nêu trên.

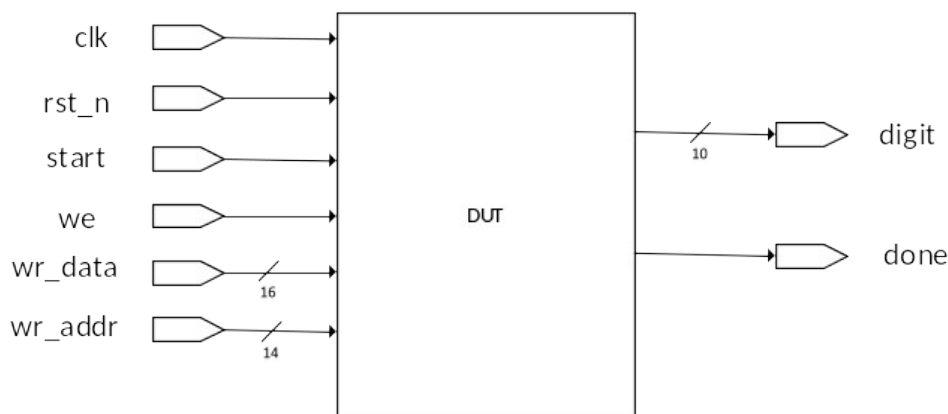
CHƯƠNG 3. THIẾT KẾ NEURAL NETWORK TRÊN FPGA

Kết thúc Chương 3, ta đã có một Neural Network trên Matlab với độ chính xác trên 90%. Mục tiêu của chương này là triển khai mạng đó trên FPGA, với thiết bị FPGA là kit DE2 – EP2C35F672C6 của hãng Altera. Không chỉ tập trung vào việc xây dựng, thiết kế còn được tập trung vào việc kiểm chứng cẩn thận để đảm bảo độ tin cậy, chi tiết sẽ được trình bày qua các phần dưới đây.

3.1. Yêu cầu kỹ thuật

3.1.1. Đầu vào và đầu ra

DUT có các chân I/O (Input/Output) như trên Hình 3.1.



Hình 3.13 Các chân I/O của DUT

Chức năng của từng chân được trình bày sơ lược trên Bảng 3.1. Thiết kế có 6 đầu vào và 2 đầu ra.

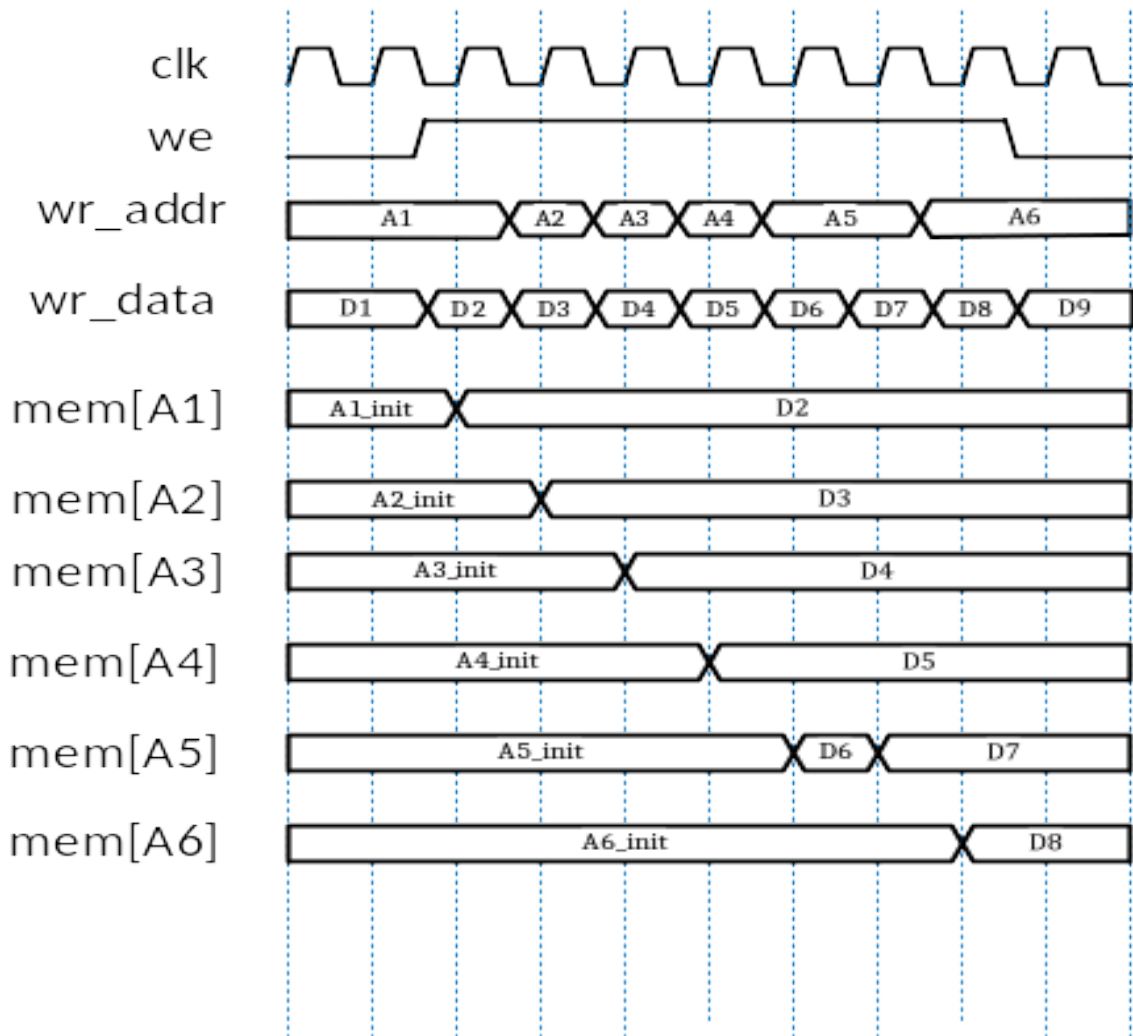
Bảng 3.3 Chức năng của các chân I/O

Tín hiệu	I/O	Kích thước (bit)	Chức năng
clk	Input	1	Xung clock để đồng bộ
rst_n	Input	1	Reset bất đồng bộ mức thấp
start	Input	1	Bắt đầu quá trình nhận dạng
we	Input	1	Cho phép ghi vào bộ nhớ
wr_data	Input	16	Giá trị sẽ ghi vào bộ nhớ
wr_addr	Input	14	Địa chỉ trên bộ nhớ mà giá trị của wr_data sẽ ghi vào
digit	Output	10	Kết quả chữ số nhận dạng được
done	Output	1	Báo hiệu đã nhận dạng xong

3.1.2. Hoạt động

Khi `rst_n` xuống mức 0 thì tất cả các đầu ra bằng 0 trong suốt quá trình `rst_n` được giữ ở mức 0. Hoạt động của hệ thống được chia thành hai hoạt động riêng biệt, một là ghi vào bộ nhớ, hai là nhận dạng.

Trong hệ thống có bộ nhớ để lưu một bức ảnh cần nhận dạng và các tham số (weight và biase) cần thiết cho việc nhận dạng. Trước khi ghi vào bộ nhớ, ảnh phải được tiền xử lý như ở Chương 3 để trở thành vector có 717 số. Hoạt động ghi vào bộ nhớ được minh họa bởi Hình 3.2.



Hình 3.14 Hoạt động ghi vào bộ nhớ

Hoạt động ghi vào bộ nhớ chỉ diễn ra khi `we = 1`. Khi `we = 0` thì bộ nhớ giữ nguyên giá trị hiện có của nó. Khi có sườn lên của `clk`, giá trị của bộ nhớ tại địa chỉ `wr_addr` được cập nhật giá trị mới bằng giá trị của `wr_data` tại thời điểm đó. `A1_init`, `A2_init`, `A3_init`, `A4_init`, `A5_init`, `A6_init` là các giá trị khởi tạo của bộ nhớ.

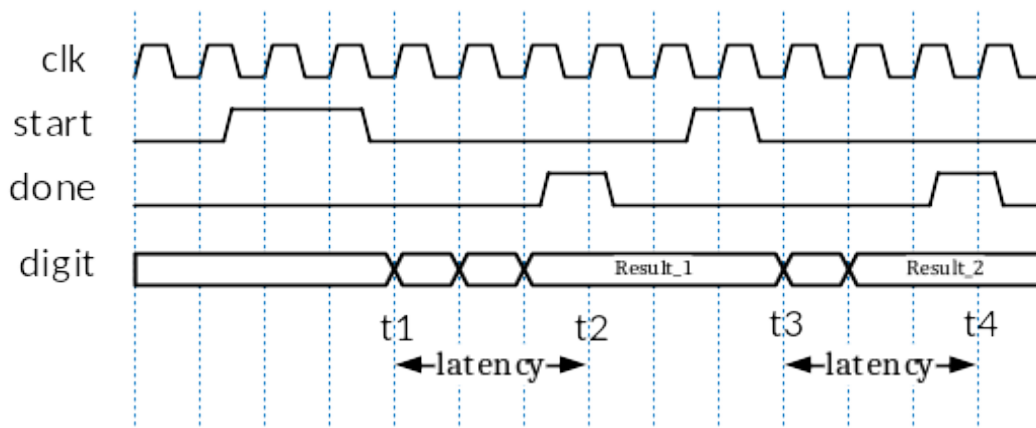
Hoạt động nhận dạng chỉ diễn ra khi $we = 0$. Khi $start = 1$ rồi trở về 0 thì bắt đầu hoạt động nhận dạng. Hoạt động nhận dạng kết thúc khi tín hiệu $done = 1$ trong 1 chu kỳ. Khi đó giá trị $digit$ là kết quả nhận dạng ảnh đã được lưu trong bộ nhớ. Bảng 3.2 cho biết đầu ra bắt buộc ứng với mỗi chữ số. Như trong Bảng 3.2 ta thấy, ví dụ đầu ra là 0000000100 có nghĩa kết quả nhận dạng ra chữ số 3. Nếu kết quả không ra giá trị nào liệt kê trong bảng thì tức là thiết kế sai. Cách mã hóa này tương tự với cách mã hóa giám sát viên trên Matlab đã trình bày trong Chương 3.

Bảng 3.4 Chữ số biểu thị ứng với đầu ra

Kết quả digit	Chữ số biểu thị
1000000000	9
0100000000	8
0010000000	7
0001000000	6
0000100000	5
0000010000	4
0000001000	3
0000000100	2
0000000010	1
0000000001	0

Trong quá trình nhận dạng thì đầu ra $digit$ được phép thay đổi giá trị. Khi tín hiệu $done = 1$ và chưa bắt đầu nhận dạng lần tiếp theo thì $digit$ không được phép thay đổi.

Chi tiết của hoạt động nhận dạng thể hiện trên Hình 3.3. Ta có $t1$ và $t3$ là thời điểm sườn lên clk bắt được sự kiện $start$ trở về 0, $t2$ và $t4$ là thời điểm sườn lên clk bắt được sự kiện $done$ lên 1. Khoảng thời gian từ $t1$ đến $t2$, và khoảng thời gian từ $t3$ đến $t4$ coi là trễ xử lý (latency), đo bằng số chu kỳ, thể hiện hoạt động nhận dạng mất bao nhiêu chu kỳ clk . Trong khoảng thời gian từ $t2$ đến $t3$, giá trị của $digit$ chính là đáp số biểu thị chữ số đã nhận dạng được, và không được phép thay đổi.



Hình 3.15 Hoạt động nhận dạng chữ số

Hệ thống phải nhận dạng được với độ chính xác trên 90% đối với tập kiểm tra MNIST, cách tính độ chính xác giống như trình bày tại Chương 3. Sau khi thiết kế thì hệ thống phải triển khai được trên kit DE2 – 2C35F672C6N của hãng Altera.

3.2. Kế hoạch kiểm chứng

Vì việc kiểm chứng phải thực hiện song song cùng với xây dựng nên ngay sau khi có yêu cầu kỹ thuật, ta phải phát triển kế hoạch kiểm chứng bám sát vào yêu cầu kỹ thuật. Dựa vào yêu cầu kỹ thuật của hệ thống đã trình bày ở trên, ta đưa ra các đặc tính sau để kiểm chứng và phương pháp kiểm chứng các đặc tính đó.

- Đặc tính 1: Khi $rst_n = 0$ thì đầu ra bằng 0, phương pháp kiểm chứng là quan sát sóng thủ công bằng mắt.
- Đặc tính 2: Khi $we = 1$ thì bộ nhớ tại địa chỉ wr_addr được ghi giá trị bằng wr_data .
- Đặc tính 3: Khi $we = 0$ thì bộ nhớ giữ nguyên giá trị hiện có.
- Đặc tính 4: Tất cả các mẫu trong tập kiểm tra của MNIST đều phải được kiểm tra với độ chính xác trên 90%.
- Đặc tính 5: Khi $we = 0$, tiến hành hoạt động nhận dạng, từ khi nhận dạng xong ($done = 1$) đến lúc $start$ trở về 0 thì giá trị của $digit$ không được thay đổi.
- Đặc tính 6: Thiết kế nạp được trên kit DE2 – EP2C35F672C6.

Các đặc tính 2, 3, 4, 5 được kiểm chứng theo một kịch bản gồm các bước sau:

Bước 1: Ghi các giá trị của các điểm ảnh của tất cả các ảnh trong tập kiểm tra của MNIST ra các file text, ta được một tập hợp các file text.

Bước 2: Lấy file text đầu tiên trong tập hợp làm file text hiện tại.

Bước 3: Reset hệ thống.

Bước 4: Ghi các giá trị của file text hiện tại vào bộ nhớ (coi như ghi ảnh cần nhận dạng vào bộ nhớ).

Bước 5: Tiến hành hoạt động nhận dạng.

Bước 6: Nếu file text hiện tại là file text cuối cùng thì dừng, nếu không phải thì lấy file text tiếp theo làm file text hiện tại, rồi chuyển sang Bước 3.

Như vậy kịch bản trên sẽ tạo vòng lặp duyệt qua tất cả các mẫu trong tập kiểm tra MNIST. Với mỗi lần lặp ta ghi lại kết quả digit xem lần đó nhận dạng đúng hay nhận dạng sai, cuối cùng thống kê lại. Đặc tính 2, 3 và 4 được kiểm tra bằng cách thống kê độ nhận dạng chính xác có đạt trên 90% hay không, nếu đạt trên 90% thì chứng tỏ đặc tính 2, 3, và 4 đúng. Khi đó, Đặc tính 2 đúng vì nếu ghi vào bộ nhớ sai thì nhận dạng chữ số lưu trong bộ nhớ cũng sẽ sai, không thể đạt độ chính xác cao trên 90% được. Khi đó Đặc tính 3 đúng vì nếu trong quá trình nhận dạng mà bộ nhớ bị thay đổi thì kết quả cũng sẽ sai, không thể đạt độ chính xác cao trên 90% được. Khi đó Đặc tính 4 đúng vì ta đã duyệt hết tất cả các mẫu trong tập kiểm tra MNIST.

Đặc tính 1 được quan sát thủ công tại Bước 2.

Đặc tính 5 được kiểm tra tại mỗi sườn lên clk giữa mỗi lần hoàn thành nhận dạng và bắt đầu nhận dạng lần tiếp theo.

Trễ xử lý tại mỗi lần nhận dạng được tính tại Bước 5.

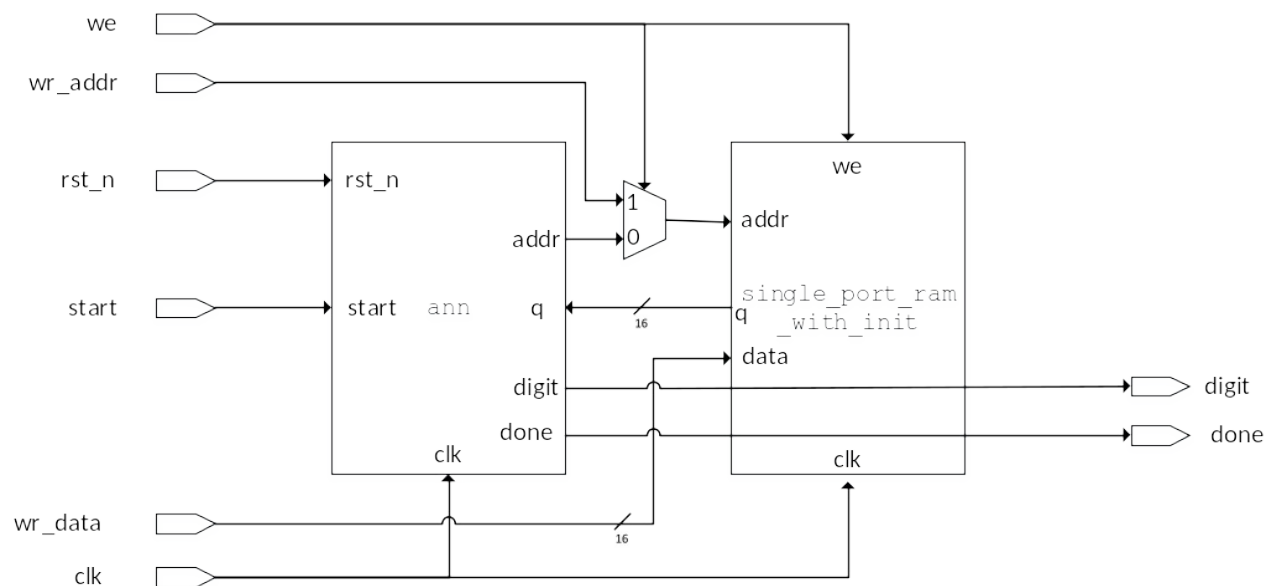
Đặc tính 6 cần được kiểm tra bằng cách biên dịch (compile) mã nguồn RTL bằng phần mềm Quartus của Altera với thiết bị đích là kit DE2 – EP2C35F672C6. Nếu biên dịch thành công thì chứng tỏ Đặc tính 6 thỏa mãn, nếu biên dịch thất bại thì chứng tỏ Đặc tính 6 không thỏa mãn.

Công cụ phục vụ cho việc kiểm chứng thiết kế là phần mềm QuestaSim của hãng Synopsys trên hệ điều hành Centos.

3.3. Thiết kế RTL

3.3.1. Khối DUT

Khối DUT là module cao nhất trong thiết kế RTL có thể tổng hợp được. Dựa vào thuật toán đã triển khai trên Matlab và yêu cầu kỹ thuật, ta chia DUT thành 2 module con như trên Hình 3.4.



Hình 3.16 Khối DUT

Khối DUT bao gồm 2 khối con là ann và single_port_ram_with init.

Khối single_port_ram_with init là ram lưu giá trị các điểm ảnh của một bức ảnh và các tham số cần thiết để nhận dạng bức ảnh đó. Bảng 3.3 thể hiện chức năng của từng dải địa chỉ trong ram.

Bảng 3.5 Ý nghĩa các dải địa chỉ trong ram

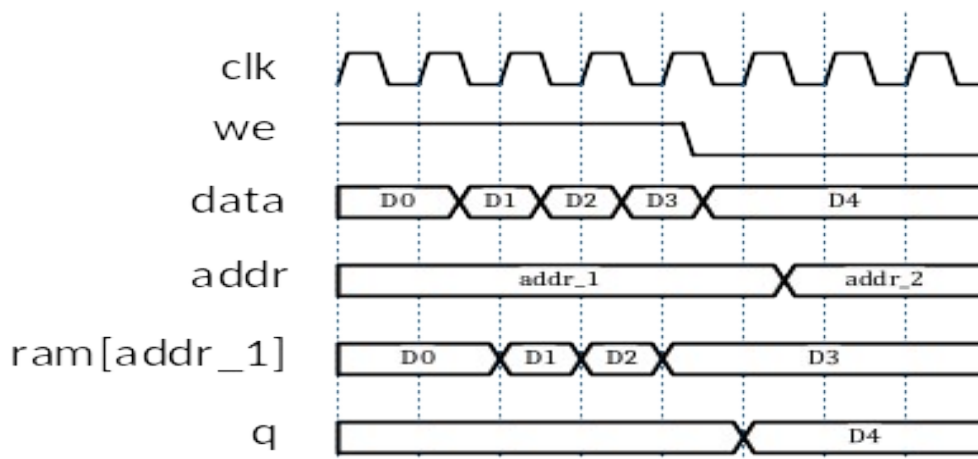
Địa chỉ	Ý nghĩa
0 – 10751	W2
10752 – 11468	K
11469 – 11483	B2
11484 – 11633	W3
11634 – 11643	B3

Các giá trị W2, B2, W3, B3 được khởi tạo sẵn trong ram, đó là giá trị mặc định. Còn K là giá trị các điểm ảnh của ảnh sau khi đã được tiền xử lý.

Bảng 3.6 Các chân I/O của single_port_ram_with_init

Chân	I/O	Độ rộng (bit)	Ý nghĩa
clk	Input	1	Xung clock để đồng bộ
we	Input	1	Quyết định ghi vào hay đọc ra tại địa chỉ addr
addr	Input	14	Địa chỉ ghi vào hoặc đọc ra
data	Input	16	Dữ liệu sẽ ghi vào
q	Output	16	Dữ liệu sẽ đọc ra

Theo Bảng 3.4, chân we quyết định tại một thời điểm sẽ chỉ ghi vào hoặc chỉ đọc ra tại địa chỉ addr. Nếu we = 1 thì ghi vào, nếu we = 0 thì đọc ra, đồng bộ theo xung clk như trên Hình 3.5.

**Hình 3.17 Ghi vào và đọc ra với single_port_ram_with_init**

Ta thấy single_port_ram_with_init có cả hai chức năng là ghi và đọc, chức năng ghi cho phép người dùng ghi vào bộ nhớ như yêu cầu kỹ thuật, chức năng đọc cho phép tải ảnh từ ram vào khối ann để tiến hành hoạt động nhận dạng. Bộ mux để lựa chọn địa chỉ hợp lý cho quá trình ghi hay đọc, nếu we = 1 thì addr của single_port_ram_with_init bằng với wr_addr để ghi vào, nếu we = 0 thì addr của single_port_ram_with_init bằng với addr của ann để đọc ra các điểm ảnh mà ann cần.

Khối ann làm nhiệm vụ nhận dạng, có các chân I/O như Bảng 3.5.

Bảng 3.7 Các chân I/O của ann

Chân	I/O	Độ rộng (bit)	Ý nghĩa
clk	Input	1	Xung clock đồng bộ
rst_n	Input	1	Reset bất đồng bộ mức thấp

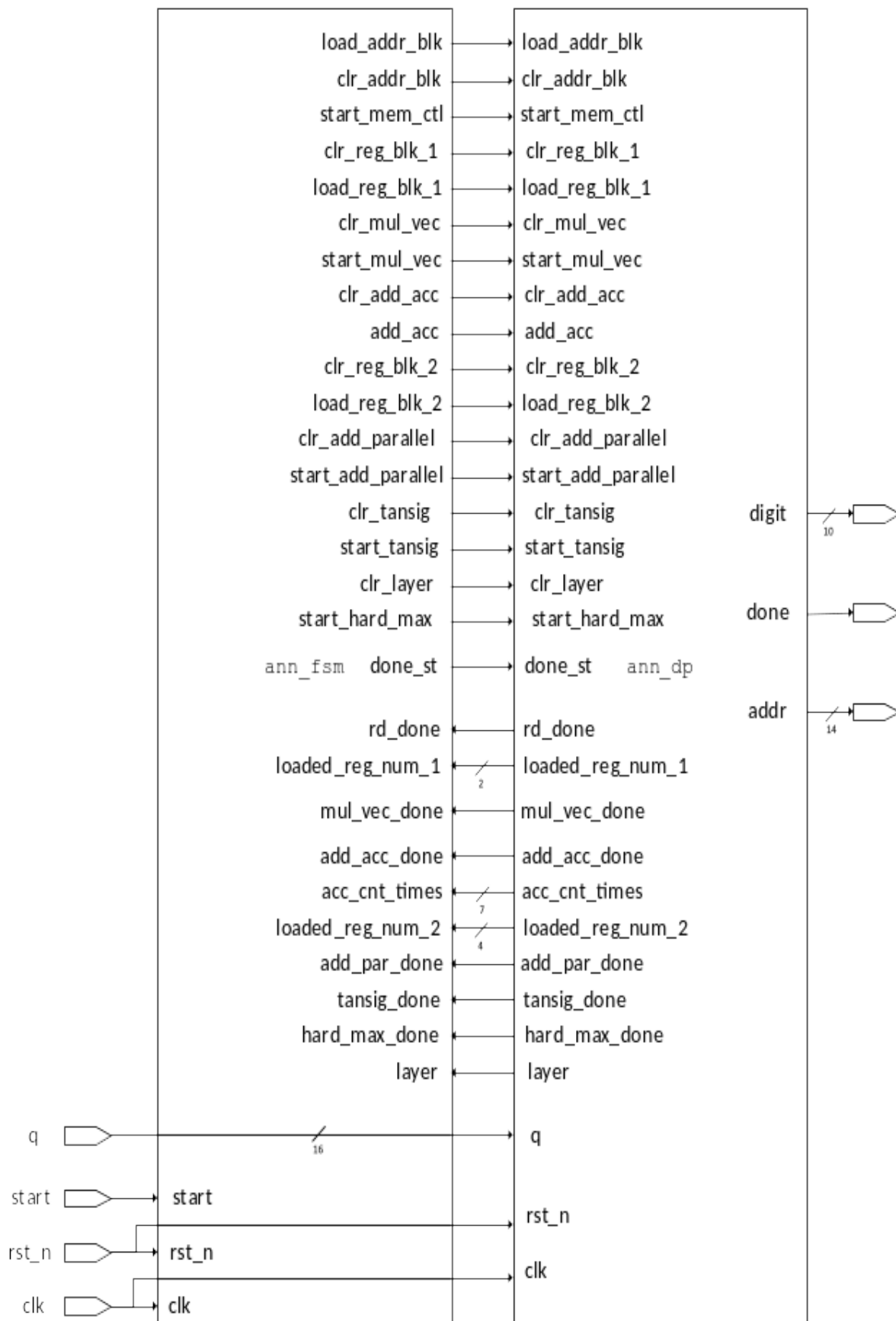
start	Input	1	Bắt đầu hoạt động nhận dạng
q	Input	16	Giá trị của một điểm ảnh cần đọc
addr	Output	1	Địa chỉ của điểm ảnh cần đọc
digit	Output	1	Kết quả chữ số nhận dạng được
done	Output	1	Báo hiệu hoạt động nhận dạng kết thúc

Khi $we = 0$, khối ann sẽ điều chỉnh addr để lấy giá trị điểm ảnh cần thiết qua q để xử lý.

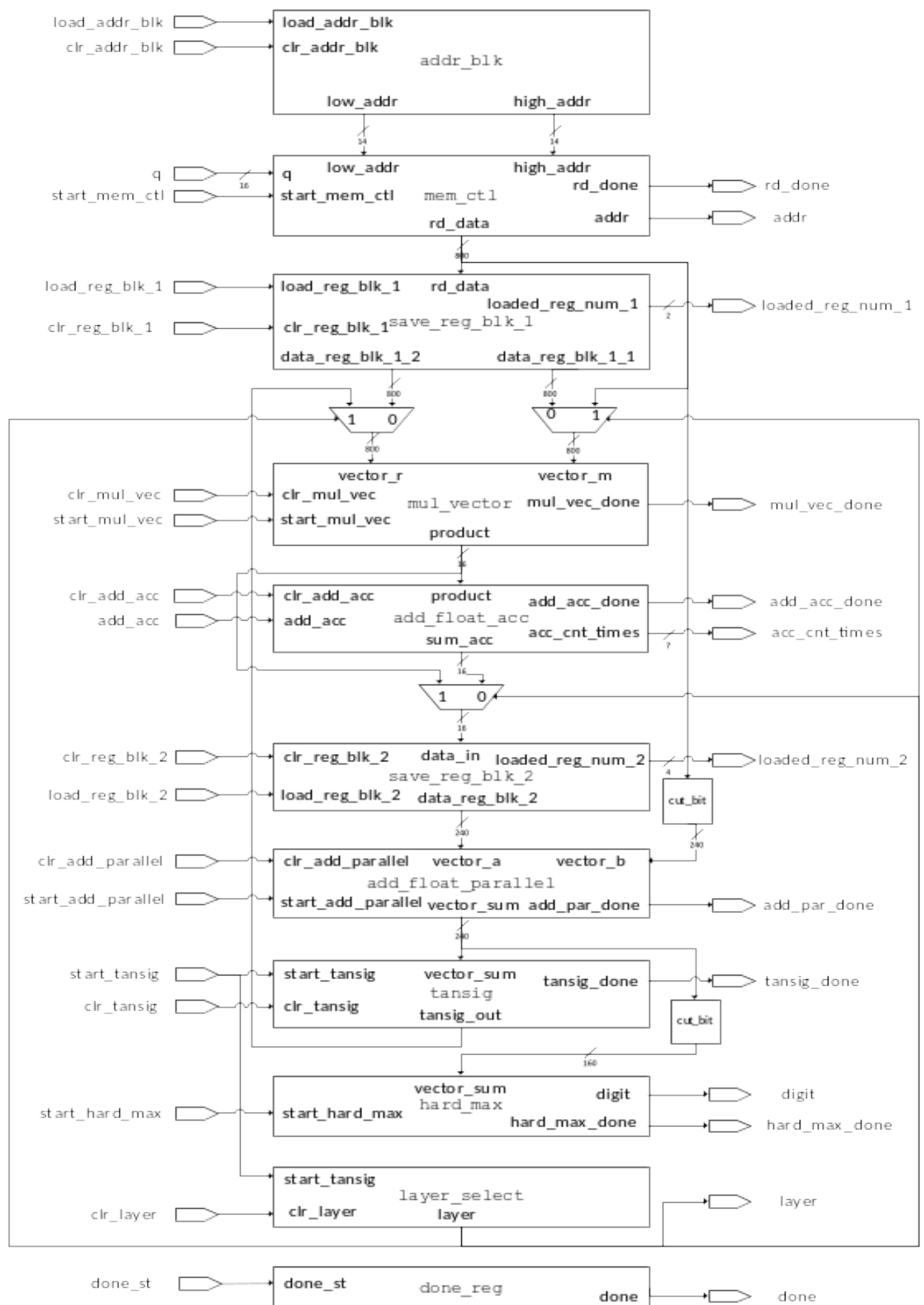
3.3.2. Khối ann

3.3.2.1 Tổng quát về khối ann

Khối ann thiết kế theo mô hình FSMD với sơ đồ khối như Hình 3.6 và Hình 3.7. Khối ann_fsm đóng vai trò control path, còn khối ann_dp đóng vai trò làm data path. Theo nguyên lý FSMD đã trình bày trong phần 1.4 thì control path và data path giao tiếp với nhau bằng các tín hiệu control signal và các tín hiệu internal status. Hình 3.7 thể hiện khối ann_dp, tuy không vẽ nhưng ta mặc định các khối đều có input là rst_n và clk.



Hình 3.18 Khối ann



Hình 3.19 Khối ann_dp

Trên Hình 3.6 ta có q là chân để ann lấy số từ `single_port_ram_with_init`, và `addr` là chân để ann cho `single_port_ram_with_init` biết lấy số ở địa chỉ nào. Để đơn giản thì có thể hình dung dữ liệu qua khối ann đi theo chiều từ q đến `digit`. Khối `addr_blk` sẽ tính toán `low_addr` và `high_addr` thích hợp tại mỗi thời điểm, khối `mem_ctl` sẽ điều chỉnh `addr` thay đổi trong khoảng từ `low_addr` đến `high_addr` để đọc ra số liệu từ `single_port_ram_with_init`. Dữ liệu từ `single_port_ram_with_init` đi từ q đến `rd_data`, rồi tiếp tục được vận chuyển đến các khối khác để thực hiện tính toán. Mỗi một lần đọc ra tối đa là 50 số nên `low_addr` và `high_addr` chênh nhau 50 đơn vị, và độ rộng của `rd_data` là $16 \times 50 = 800$ bit.

Về mặt toán học, khối ann sẽ làm hoạt động nhận dạng bằng việc thực hiện các phép toán sau đây:

$$Z2 = W2 \cdot K + B2 \quad (3.1)$$

$$A2 = A = \text{tansig}(Z2) \quad (3.2)$$

$$Z3 = W3 \cdot A2 + B3 \quad (3.3)$$

$$A3 = \text{hard}_{\max}(Z3) \quad (3.4)$$

với:

- K cỡ 717×1
- $W2$ cỡ 15×717
- $B2$ và $Z2$ cỡ 15×1
- $W3$ cỡ 10×15
- $B3$ và $Z3$ cỡ 10×1
- $A3$ cỡ 10×1

3.3.2.2 Tính công thức (3.1)

Trong công thức (3.1) có phép nhân $W2$ (cỡ 15×717) với K (cỡ 717×1) là phép nhân hai ma trận có kích thước lớn, nếu nhân song song các phần tử thì sẽ rất tốn tài nguyên, nên phải chia nhỏ hai ma trận trên để thực hiện phép nhân hai vector kích thước 50×1 với nhau rồi cộng lại. Cụ thể như công thức (3.5).

$$\begin{bmatrix} w_{1,1}^2 & \dots & w_{717,1}^2 \\ \dots & \dots & \dots \\ w_{1,15}^2 & \dots & w_{717,15}^2 \end{bmatrix} \begin{bmatrix} K_1 \\ \dots \\ K_{717} \end{bmatrix} = \begin{bmatrix} \begin{bmatrix} w_{1,1}^2 & \dots & w_{50,1}^2 \\ \dots & \dots & \dots \\ w_{1,15}^2 & \dots & w_{50,15}^2 \end{bmatrix} \begin{bmatrix} K_1 \\ \dots \\ K_{50} \end{bmatrix} + \begin{bmatrix} w_{51,1}^2 & \dots & w_{100,1}^2 \\ \dots & \dots & \dots \\ w_{51,15}^2 & \dots & w_{100,15}^2 \end{bmatrix} \begin{bmatrix} K_{51} \\ \dots \\ K_{100} \end{bmatrix} + \dots + \begin{bmatrix} w_{701,1}^2 & \dots & w_{750,1}^2 \\ \dots & \dots & \dots \\ w_{701,15}^2 & \dots & w_{750,15}^2 \end{bmatrix} \begin{bmatrix} K_{701} \\ \dots \\ K_{750} \end{bmatrix} \\ \dots \\ \begin{bmatrix} w_{1,15}^2 & \dots & w_{50,15}^2 \\ \dots & \dots & \dots \\ w_{1,15}^2 & \dots & w_{50,15}^2 \end{bmatrix} \begin{bmatrix} K_1 \\ \dots \\ K_{50} \end{bmatrix} + \begin{bmatrix} w_{51,15}^2 & \dots & w_{100,15}^2 \\ \dots & \dots & \dots \\ w_{51,15}^2 & \dots & w_{100,15}^2 \end{bmatrix} \begin{bmatrix} K_{51} \\ \dots \\ K_{100} \end{bmatrix} + \dots + \begin{bmatrix} w_{701,15}^2 & \dots & w_{750,15}^2 \\ \dots & \dots & \dots \\ w_{701,15}^2 & \dots & w_{750,15}^2 \end{bmatrix} \begin{bmatrix} K_{701} \\ \dots \\ K_{750} \end{bmatrix} \end{bmatrix} \quad (3.5)$$

Để thực hiện phép tính nhân W2 với K, mỗi lần ta đọc từ single_port_ram_with_init ra 50 số của W2, lưu 50 số đó trong thanh ghi data_reg_blk_1_1, rồi đọc tiếp 50 số của K, lưu 50 số đó trong thanh ghi data_reg_blk_1_2. Lúc này khối mul_vec sẽ nhân hai vector data_reg_blk_1_1 và data_reg_blk_1_2 được tích là product. Các product này sẽ được cộng tích lũy trong khối add_float_acc. Vì K có độ dài 717, mỗi lần đọc ra 50 số để nhân, nên khối mul_vec và khối add_float_acc sẽ hoạt động $\left\lceil \frac{717}{50} \right\rceil + 1 = 15$ lần thì kết quả sum_acc sẽ được lưu vào data_reg_blk_2. Vì W2 có 15 hàng nên sau khi lưu được 15 lần thì tức là đã thực hiện xong phép nhân W2 với K, kết quả lúc này là 15 số lưu trong thanh ghi data_reg_blk_2. Mỗi số có độ rộng 16 bit nên thanh ghi data_reg_blk_2 có độ rộng là $16 \times 15 = 240$ bit.

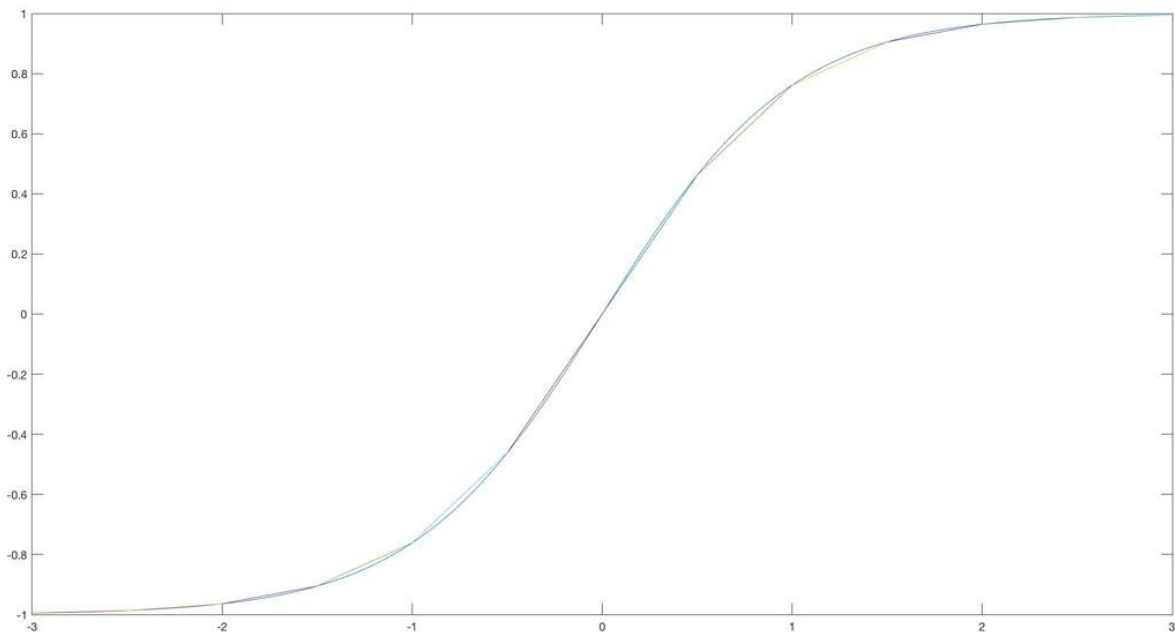
Sau khi nhân xong W2 với K, ta cộng kết quả có được với B2 bằng khối add_float_parallel. Khối add_float_parallel cộng hai vector với nhau bằng cách cộng song song các phần tử tương ứng. Vector B2 được đọc ra từ single_port_ram_with_init, bởi rd_data là thanh ghi lưu 50 số nên ta cần khối cut_bit để lấy ra 15 số đưa vào add_float_parallel.

3.3.2.3 Tính công thức (3.2)

Kết quả của công thức (3.1) nằm trong thanh ghi vector_sum, thanh ghi này được nối vào khối tansig để thực hiện hàm tansig. Khối tansig tính tansig của một ma trận bằng cách tính tansig của mỗi phần tử đồng thời. Phương pháp tính là tuyến tính hóa hàm tansig. Thực tế đồ thị hàm tansig là một đường cong, nhưng có thể xấp xỉ thành đường gấp khúc như công thức (3.6).

$$y = \text{tansig}(x) = \frac{2}{1 + e^{-2x}} - 1 \approx \begin{cases} -1 & (\text{nếu } x \leq -3) \\ 0.016881x - 0.94441 & (\text{nếu } -3 \leq x < -2.5) \\ 0.045173x - 0.87368 & (\text{nếu } -2.5 \leq x < -2) \\ 0.11776x - 0.72851 & (\text{nếu } -2 \leq x < -1.5) \\ 0.28711x - 0.47449 & (\text{nếu } -1.5 \leq x < -1) \\ 0.59895x - 0.16264 & (\text{nếu } -1 \leq x < -0.5) \\ 0.92423x & (\text{nếu } -0.5 \leq x < 0.5) \\ 0.59895x + 0.16264 & (\text{nếu } 0.5 \leq x < 1) \\ 0.28711x + 0.47449 & (\text{nếu } 1 \leq x < 1.5) \\ 0.11776x + 0.72851 & (\text{nếu } 1.5 \leq x < 2) \\ 0.045173x + 0.87368 & (\text{nếu } 2 \leq x < 2.5) \\ 0.016881x + 0.94441 & (\text{nếu } 2.5 \leq x < 3) \\ 1 & (\text{nếu } x \geq 3) \end{cases} \quad (3.6)$$

Đường cong trên Hình 3.8 là đồ thị của hàm tansig thật sự, còn đường gấp khúc là đồ thị của hàm tansig sau khi tuyến tính hóa như công thức (3.6), sự sai lệch giữa đường cong và đường gấp khúc là rất nhỏ, hai đường này gần như trùng nhau.



Hình 3.20 Hàm tansig thật sự và hàm tansig đã tuyến tính hóa

Sau khi tuyến tính hóa thì việc tính hàm tansig sẽ đơn giản hơn rất nhiều, từ phép tính lũy thừa với số mũ e rất phức tạp, ta chỉ phải thực hiện phép so sánh, phép nhân và phép cộng. Đó là phương pháp được lựa chọn để triển khai trên FPGA, chấp nhận sai số.

3.3.2.4 Tính công thức (3.3)

Tại công thức (3.3), ta phải thực hiện phép nhân W3 (cỡ 10 x 15) với A2 (cỡ 15 x 1). Vì vector A2 chỉ có 15 phần tử nên không cần phải tách thành các vector nhỏ hơn để nhân giống như công thức (3.1). Lúc này ta không cần khối cộng tích lũy add_float_acc nữa. Bộ nhân mul_vec được thiết kế để nhân 2 vector có độ dài 50 nên có thể nhân được 2 vector có độ dài là 15. Kết quả nhân W3 với A2 được lưu trong data_reg_blk_2, sau đó cộng với B3 tại khối add_float_parallel. Vector B3 được đọc ra từ single_port_ram_with_init. Lưu ý là khối mul_vec dùng để nhân ở cả công thức (3.1) và công thức (3.3) nên cần có các bộ mux để lựa chọn đầu vào. Nếu việc tính toán đang ở lớp 2 (layer = 0) thì tức là đang cần tính công thức (3.1), còn nếu việc tính toán đang ở lớp 3 (layer = 1) thì tức là đang cần tính công thức (3.3).

3.3.2.5 Tính công thức (3.4)

Khối hard_max sẽ tính công thức (3.4), cụ thể như công thức (3.7).

$$A3 = 1 \ll i \text{ với } Z3[i] \text{ là phần tử lớn nhất trong } Z3 \quad (3.7)$$

3.3.2.6 Control signal và internal status

Nhóm các tín hiệu đóng vai trò là control signal trong khối ann được trình bày trong Bảng 3.6.

Bảng 3.8 Các tín hiệu control signal trong khối ann

Tên tín hiệu	Ý nghĩa
load_addr_blk	Tính ra low_addr và high_addr.
clr_addr_blk	Xóa giá trị low_addr và high_addr.
start_mem_ctl	Bắt đầu đọc các giá trị trong single_port_ram_with_init từ địa chỉ low_addr đến địa chỉ high_addr vào thanh ghi rd_data. Đầu vào q có độ rộng 16 bit, dải địa chỉ từ low_addr đến high_addr rộng 50 địa chỉ nên thanh ghi rd_data có độ rộng là $16 \times 50 = 800$ bit.
clr_reg_blk_1	Xóa giá trị lưu trong các thanh ghi của khối save_reg_blk_1
load_reg_blk_1	Cho phép các thanh ghi trong khối save_reg_blk_1 lưu các giá trị rd_data, để sau đó thực hiện phép nhân vector.
clr_mul_vec	Xóa giá trị lưu trong các thanh ghi của khối mul_vector.
start_mul_vec	Bắt đầu nhân hai vector.
clr_add_acc	Xóa giá trị lưu trong các thanh ghi của khối add_float_acc.
add_acc	Cho phép cộng giá trị tiếp theo vào giá trị đã tích lũy được.
clr_reg_blk_2	Xóa giá trị lưu trong các thanh ghi của khối save_reg_blk_2
load_reg_blk_2	Cho phép các thanh ghi trong khối save_reg_blk_2 lưu các giá trị vừa tính được từ khối add_float_acc.
clr_add_parallel	Xóa giá trị lưu trong các thanh ghi của khối add_float_parallel.

start_add_parallel	Bắt đầu cộng hai vector, các phần tử tương ứng được cộng cùng thời điểm.
clr_tansig	Xóa giá trị lưu trong các thanh ghi của khối tansig.
start_tansig	Bắt đầu tính tansig.
clr_layer	Đưa giá trị layer về 0.
start_hard_max	Bắt đầu tính hard_max.
done_st	Thông báo cho data path biết đã hoàn thành quá trình nhận dạng.

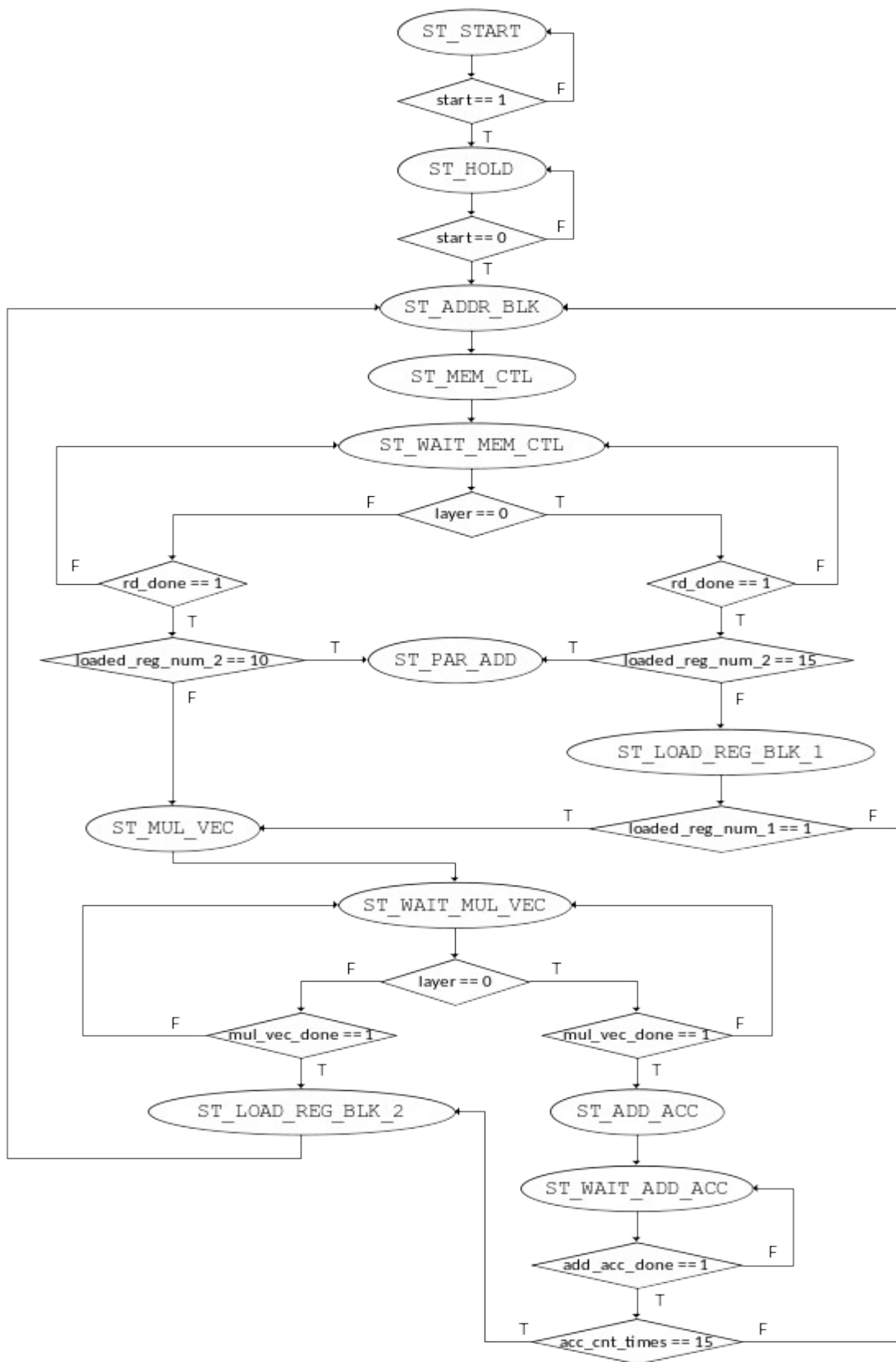
Nhóm các tín hiệu đóng vai trò là internal status trong ann được trình bày trong Bảng 3.7. Nhìn chung các tín hiệu internal status này đều làm nhiệm vụ phản hồi từ data path về cho control path biết quá trình tính toán của một khối nào đó đã hoàn thành, nhờ đó mà control path sẽ chuyển trạng thái và ra lệnh cho khối tiếp theo làm việc.

Bảng 3.9 Các tín hiệu internal status trong khối ann

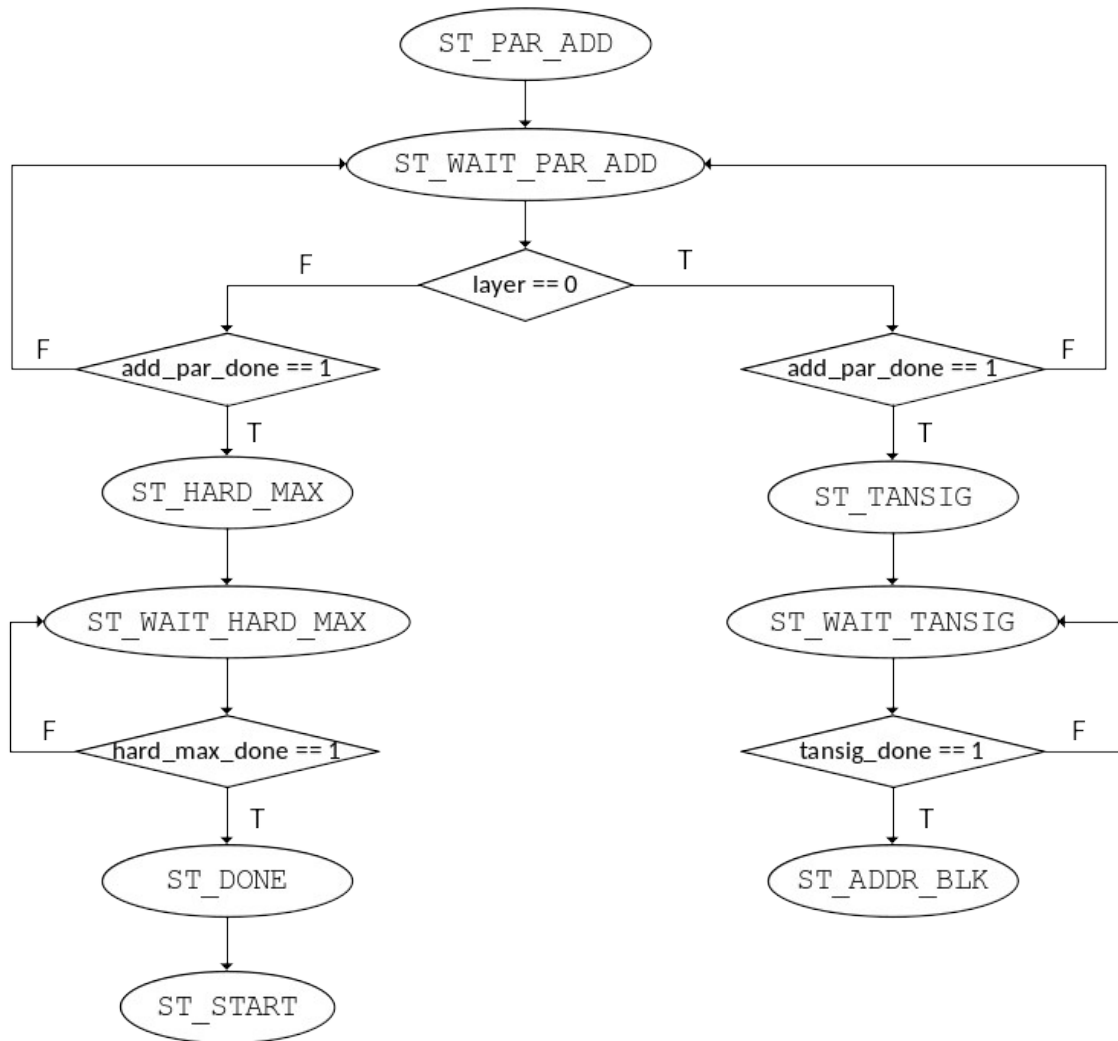
Tên tín hiệu	Ý nghĩa
rd_done	Phản hồi rằng việc đọc ra từ single_port_ram_with_init đã thành công, dữ liệu trong rd_data đã có thể sử dụng được.
loaded_reg_num_1	Phản hồi rằng khối save_reg_blk_1 có bao nhiêu thanh ghi đã được nạp giá trị từ rd_data. Vì khối save_reg_blk_1 có 2 thanh ghi có thể nhận giá trị từ rd_data nên giá trị đếm lớn nhất của loaded_reg_num_1 là 2 và độ rộng tối thiểu là 2 bit.
mul_vec_done	Phản hồi rằng khối mul_vec đã nhân xong hai vector.
add_acc_done	Phản hồi rằng khối add_float_acc đã cộng xong một số vào giá trị mà nó đã tích lũy được.
acc_cnt_times	Phản hồi rằng khối add_float_acc đã cộng tích lũy được bao nhiêu lần. Như đã trình bày trên thì khối add_float_acc sẽ cộng tích lũy 15 lần nên giá trị đếm lớn nhất của acc_cnt_times là 15 và độ rộng tối thiểu là 4 bit.
add_par_done	Phản hồi rằng khối add_float_parallel đã cộng xong hai vector.
tansig_done	Phản hồi rằng khối tansig đã tính xong hàm tansig của vector đầu vào.
hard_max_done	Phản hồi rằng khối hard_max đã tính xong hàm hard_max, kết quả digit có thể sử dụng được.
layer	Phản hồi việc tính toán đang ở lớp thứ mấy. Nếu layer = 0 thì việc tính toán đang ở lớp lớp ẩn, tức là đang tính công thức (3.1) và (3.2). Nếu layer = 1 thì việc tính toán đang ở lớp lớp đầu ra, tức là đang tính công thức (3.3) và (3.4).

3.3.2.7 Sơ đồ FSM

Sơ đồ FSM của control path sẽ diễn tả chi tiết hơn về việc phối hợp hoạt động của các khối con trong data path.



Hình 3.21 Sơ đồ FSM của ann_fsm



Hình 3.22 Sơ đồ FSM của `ann_fsm` (tiếp tục)

Trên Hình 3.9 và Hình 3.10 là sơ đồ FSM của khối `ann_fsm`, với các cặp state trùng tên trên hai hình (`ST_START`, `ST_PAR_ADD`, `ST_ADDR_BLK`) là chỉ cùng một trạng thái. Ta coi khi `rst_n` bằng 0 thì dù ở trạng thái nào mạch cũng ngay lập tức quay về trạng thái `ST_START`, nếu `rst_n` bằng 1 thì tại sườn lên `clk`, mạch sẽ chuyển sang trạng thái tiếp theo như trên hình vẽ. Các đầu ra của `ann_fsm` là các control signal trong FSMD. Loại FSM được sử dụng là FSM loại Moore với giá trị đầu ra tại mỗi trạng thái được liệt kê trong Bảng 3.8. Ứng với mỗi trạng thái, các đầu ra được liệt kê trong cột Tín hiệu khẳng định nhận giá trị 1, các đầu ra không được liệt kê nhận giá trị 0.

Bảng 3.10 Giá trị đầu ra bằng 1 tại mỗi trạng thái của ann_fsm

Trạng thái	Công việc của ann	Tín hiệu khẳng định
ST_START	Không làm gì	
ST_HOLD	Chờ để bắt đầu tính toán	
ST_ADDR_BLK	Tính toán dải địa chỉ cần đọc ra từ single_port_ram_with_init	load_addr_blk clr_mul_vec
ST_MEM_CTL	Bắt đầu đọc dữ liệu ra từ single_port_ram_with_init	start_mem_ctl
ST_WAIT_MEM_CTL	Đợi quá trình đọc dữ liệu ra từ single_port_ram_with_init kết thúc	
ST_LOAD_REG_BLK_1	Lưu dữ liệu vừa đọc được từ single_port_ram_with_init vào các thanh ghi	load_reg_blk_1
ST_MUL_VEC	Bắt đầu nhân hai vector	start_mul_vec clr_add_parallel
ST_WAIT_MUL_VEC	Đợi quá trình nhân hai vector kết thúc	
ST_ADD_ACC	Bắt đầu cộng tích lũy tích của các cặp vector	add_acc clr_reg_blk_1
ST_WAIT_ADD_ACC	Đợi quá trình cộng tích lũy kết thúc	
ST_LOAD_REG_BLK_2	Lưu các giá trị tổng vừa cộng tích lũy hoặc các tích hai vector vào các thanh ghi	load_reg_blk_2 clr_add_acc
ST_PAR_ADD	Bắt đầu cộng hai vector	start_add_parallel clr_mul_vec
ST_WAIT_PAR_ADD	Đợi quá trình cộng hai vector kết thúc	
ST_TANSIG	Bắt đầu tính tansig của vector	start_tansig clr_reg_blk_2
ST_WAIT_TANSIG	Đợi quá trình tính tansig kết thúc	
ST_HARD_MAX	Bắt đầu tính hàm hard_max của vector	start_hard_max
ST_WAIT_HARD_MAX	Đợi quá trình tính hàm hard_max kết thúc	
ST_DONE	Kết thúc nhận dạng	done_st clr_layer clr_addr_blk clr_reg_blk_1 clr_mul_vec clr_add_acc clr_reg_blk_2 clr_add_parallel clr_tansig

Trên Hình 3.7 ta thấy khối `ann_dp` gồm nhiều khối con, mỗi khối con đó cũng được xây dựng theo mô hình FSM. Như vậy, DUT là một thiết kế gồm có nhiều tầng FSM.

3.3.3. Biểu diễn tham số của Neural Network trên FPGA

Như đã biết, Neural Network có các tham số:

- W2 là ma trận chứa các weight tại lớp ẩn, cỡ 15x717.
- B2 là ma trận chứa các bias tại lớp ẩn, cỡ 15x1.
- W3 là ma trận chứa các weight tại lớp đầu ra, cỡ 10x15.
- B3 là ma trận chứa các bias tại lớp đầu ra, cỡ 10x1

Vậy tổng cộng mạng có chứa 10930 phần tử. Khối `single_port_ram_with_init` cần phải lưu 10930 phần tử đó, cộng với 717 phần tử của ảnh cần nhận dạng, tổng là 11647 số. Để có thể lưu được 11647 số đó vào bộ nhớ trên kit DE2, ta cần phải có cách biểu diễn số phù hợp, làm sao để cân đối giữa kích thước bit và độ chính xác của số. Trước tiên, phần này sẽ trình bày về cách biểu diễn số thực theo chuẩn IEEE 754 single-precision binary floating-point format [9], còn gọi là biểu diễn số thực dấu phẩy động 32 bit theo chuẩn IEEE 754.

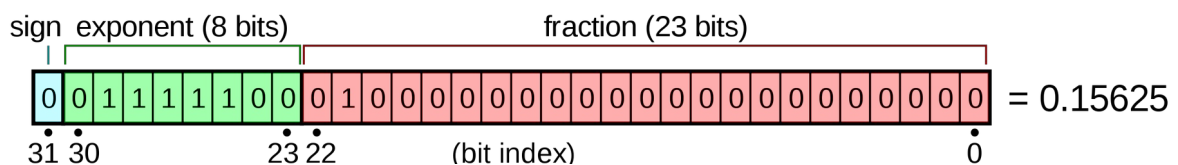
Số thực 32 bit được chia thành 3 trường theo thứ tự lần lượt từ trái qua phải.

- Sign: rộng 1 bit, ký hiệu là s .
- Exponent: rộng 8 bit, ký hiệu là e .
- Fraction: rộng 23 bit, ký hiệu là f .

Riêng đối với số 0, ta biểu diễn bởi 32 bit 0 mà không tuân theo công thức (3.8). Đối với các số khác 0, ta quy đổi theo các công thức (3.8), với v là số cần quy đổi.

$$v = (-1)^s \times 2^{e-127} \times (1.f)_2 \quad (3.8)$$

Ví dụ số +0.15625 biểu diễn theo chuẩn này sẽ được kết quả như Hình 3.11.



Hình 3.23 Kết quả biểu diễn số 0.15625 [9]

Thật vậy, ta có:

$$v = (-1)^0 \times 2^{(01111100)_2 - 127} \times (1.010 \dots 0)_2 = 1 \times 2^{124 - 127} \times 1.25 = 0.15625 \quad (3.9)$$

Dựa vào chuẩn trên, ta hình thành ý tưởng biểu diễn tham số của Neural Network trên FPGA. Ta biểu diễn số thực dưới dạng dấu phẩy động 16 bit gần giống với cách trên. Cụ thể là ta chia 16 bit thành 3 trường:

- Sign: rộng 1 bit, ký hiệu là s.
- Exponent: rộng 8 bit, ký hiệu là e.
- Fraction: rộng 7 bit, ký hiệu là f.

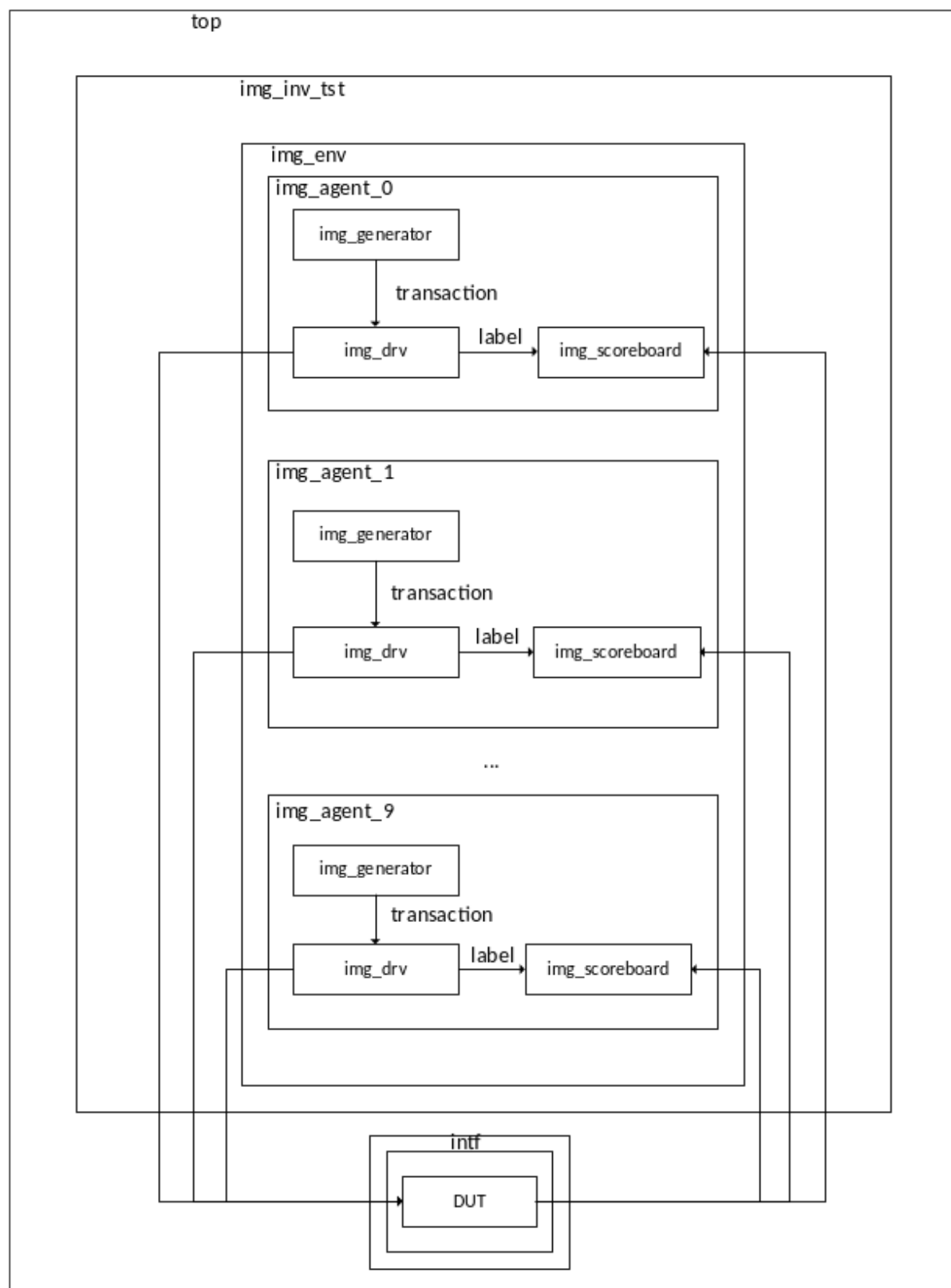
Công thức quy đổi vẫn là công thức (3.8).

Thực tế thì cách biểu diễn số trong bộ nhớ máy tính của Matlab là dấu phẩy động 64 bit, nên với cách biểu diễn bằng 16 bit như trên chắc chắn sẽ gây ra sai số giữa FPGA với Matlab. Nhưng nếu biểu diễn số bằng 64 bit như Matlab thì kit DE2 sẽ bị thiếu tài nguyên, do đó ta phải đánh đổi giữa độ chính xác và kích thước.

3.4. Thiết kế testbench

3.4.1. Kiến trúc của testbench

Testbench được thiết kế theo kế hoạch kiểm chứng đã trình bày trong phần 3.2. Về cơ bản thì testbench này là một testbench có nhiều lớp, được xây dựng dựa trên lý thuyết đã trình bày ở phần 1.5, nhưng được tùy biến một số lớp để phù hợp. Hình 3.12 là sơ đồ khối của testbench.



Hình 3.24 Sơ đồ khối testbench

Chức năng của từng khối được trình bày trong Bảng 3.9. SystemVerilog có hỗ trợ kiểu dữ liệu phù hợp cho từng khối, đó là điểm mạnh của ngôn ngữ này.

Bảng 3.11 Chức năng các khối trong testbench

Khối	Loại	Kiểu dữ liệu	Chức năng
top	top	module	là module cao nhất trong project, bao gồm DUT, interface và test.
img_inv_tst	test	program	Chứa enviroment và cấu hình agent nào trong enviroment sẽ đưa transaction vào DUT thông qua interface.
img_env	enviroment	class	Bao gồm 10 agent từ img_agent_0 đến img_agent_9.
từ img_agent_0 đến img_agent_9	agent	class	Bao gồm generator, driver và scoreboard. Mỗi agent chỉ chứa một loại chữ số. Các mẫu trong img_agent_0 có label là số 0, các mẫu trong img_agent_1 có label là số 1, ... Các mẫu trong img_agent_9 có label là số 9.
img_generator	generator	class	Tạo ra các transaction
img_drv	driver	class	Biến đổi transaction thành các tín hiệu logic để gán vào các chân input của DUT.
img_transaction	transaction	class	Chứa giá trị của các điểm ảnh của ảnh cần nhận dạng.
img_scoreboard	scoreboard	class	Lấy đầu ra từ DUT để so sánh với giá trị mong muốn.
intf	interface	interface	Nhóm các chân tín hiệu của DUT lại thành một đối tượng để DUT dễ giao tiếp với test.

Về cơ bản thì có thể coi luồng dữ liệu của img_transaction trong testbench đi từ img_generator, qua img_driver đến DUT, rồi cuối cùng đến img_scoreboard. Với mỗi img_transaction sẽ có một label tương ứng. Tại img_scoreboard sẽ so sánh label với đầu ra của DUT, nếu bằng nhau thì tức là nhận dạng đúng, nếu khác nhau thì tức là nhận dạng sai. Tại img_scoreboard cũng tiến hành kiểm tra Đặc tính 5 trong kế hoạch kiểm chứng, đó là: Khi we = 0, tiến hành hoạt động nhận dạng, từ khi nhận dạng xong (done = 1) đến lúc start trở về 0 thì giá trị của digit không được thay đổi.

Trong testbench này không có khối assertion vì yêu cầu kỹ thuật không đề cập đến mối quan hệ về thời gian giữa các tín hiệu trong DUT. Khối functional coverage tuy không có trong testbench nhưng được theo dõi thủ công theo thời gian để biết tiến độ của kiểm chứng.

3.4.2. Kết quả kiểm chứng thiết kế

Trong phần kế hoạch kiểm chứng đã nói về kịch bản hoạt động của testbench. Sau khi sử dụng kịch bản đó, tức là đã kiểm tra đối với tất cả các mẫu trong tập kiểm tra MNIST, ta được kết quả như thống kê trong Bảng 3.10.

Bảng 3.12 Kết quả kiểm chứng thiết kế

Chữ số	SUM	PASS	FAIL	% PASS	CHANGE
0	980	959	2 1	97.86	0
1	113 5	1117	1 8	98.41	0
2	103 2	963	6 9	93.31	0
3	101 0	935	7 5	92.57	0
4	982	932	5 0	94.91	0
5	892	806	8 6	90.36	0
6	958	913	4 5	95.30	0
7	102 8	966	6 2	93.97	0
8	974	892	8 2	91.58	0
9	100 9	928	8 1	91.97	0

Cột SUM là tổng số mẫu ứng với mỗi chữ số, cột PASS thể hiện có bao nhiêu mẫu được nhận dạng đúng, cột FAIL thể hiện có bao nhiêu mẫu bị nhận dạng sai.

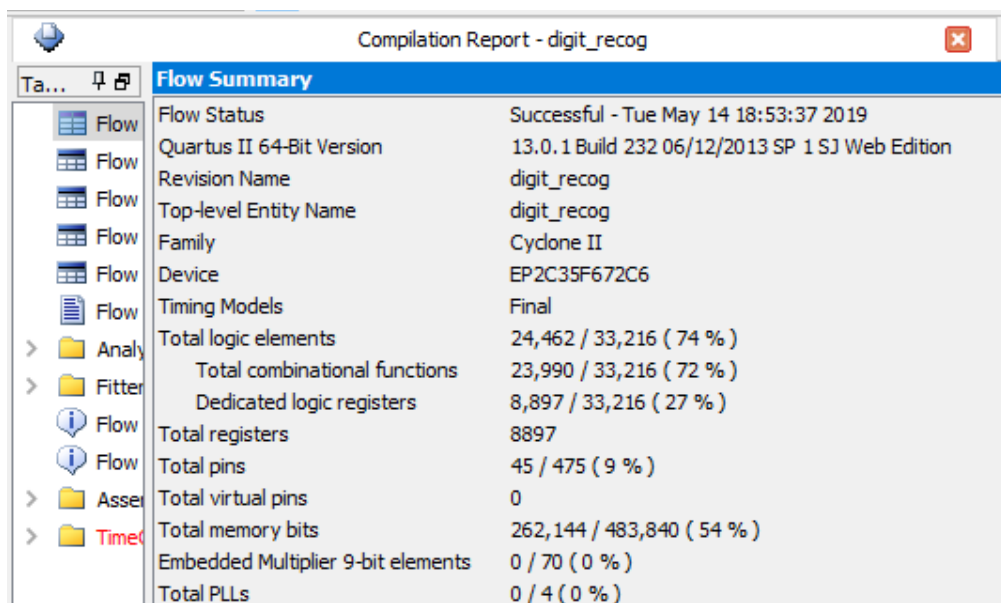
Độ chính xác trung bình bằng trung bình cộng của cột (% PASS) là 94.02%.

Độ chính xác trung bình đạt trên 90% chứng tỏ Đặc tính 2, 3 và 4 đúng.

Cột CHANGE trong bảng thể hiện có bao nhiêu mẫu vi phạm Đặc tính 5, và kết quả là không có mẫu nào bị vi phạm, vậy thiết kế thỏa mãn Đặc tính 5.

Đặc tính 1 về reset được quan sát thủ công trên sóng cũng cho kết quả đúng.

Sau khi kiểm tra thấy thỏa mãn các Đặc tính 1, 2, 3, 4 và 5, ta tiến hành biên dịch mã nguồn RTL trên phần mềm Quartus với thiết bị đích là kit DE2 – 2C35F672C6N của hãng Altera, mục đích là để kiểm tra Đặc tính 6. Kết quả là đã biên dịch thành công như Hình 3.13. Thiết kế chiếm 74% các thành phần logic và 54% bộ nhớ của kit.



Hình 3.25 Biên dịch thành công cho kit EP2C35F672C6

Vậy tất cả các Đặc tính đều thỏa mãn, thiết kế đã đạt được yêu cầu kỹ thuật, ta có độ bao phủ chức năng đã đạt được 100%.

Ngoài ra ta còn thống kê được trễ xử lý trung bình theo đơn vị là số lượng xung clk, thống kê trong Bảng 3.11.

Bảng 3.13 Trễ xử lý trung bình với từng chữ số

Chữ số	Trễ xử lý trung bình
0	99380
1	99073
2	99228
3	99189
4	99171

5	99196
6	99260
7	99122
8	99158
9	99074

Từ việc tính được trễ xử lý ta có nhận xét như sau:

Hiện nay các video thường chạy với tốc độ 30 fps (30 frames per second), tức là trong một giây có 30 hình ảnh được phát, hay tốc độ thay đổi hình ảnh là $\frac{1}{30}$ giây. Giả sử với mỗi hình ảnh ta đưa qua hệ thống này để nhận dạng chữ số theo real time. Các trễ xử lý thống kê được không có số nào vượt quá 100000 chu kỳ clk. Ngay cả khi làm tròn trễ xử lý vừa thống kê được lên thành 100000 số chu kỳ clk, thì độ dài chu kỳ tối đa để chạy được real time là: $\frac{\frac{1}{30}}{100000} = \frac{1}{3000000}$ (s), hay tần số tối thiểu để chạy được real time là 3 MHz.

Tần số 3 MHz là tần số quá thấp đối với các thiết bị FPGA trên thị trường hiện nay, thường chạy với tần số tối thiểu khoảng vài chục MHz. Do đó việc áp dụng thiết kế này vào một hệ thống nhận dạng theo real time là khả thi về mặt công nghệ.

3.5. Kết luận chương

Thiết kế Neural Network trên FPGA đã thỏa mãn tất cả các yêu cầu kỹ thuật. So với độ chính xác 94.08% trên Matlab, độ chính xác trên FPGA đạt thấp hơn một chút là 94.02%, tuy nhiên vẫn đạt trên 90% như yêu cầu đề ra. Sự khác nhau này là do cách biểu diễn tham số của Matlab là 64 bit, còn cách biểu diễn tham số trên FPGA là 16 bit. Độ chính xác nêu trên là độ chính xác đối với tập kiểm tra MNIST, là một tập dữ liệu cụ thể chứ không phải là độ chính xác theo cách nói chung chung không có căn cứ cụ thể, do đó người dùng có thể hình dung được sự tin cậy của thiết kế này.

Kiến trúc RTL của thiết kế được xây dựng dựa theo mô hình FSMĐ gồm nhiều tầng. Tầng cao nhất của thiết kế gồm một control path và một data path, trong đó data path lại được xây dựng từ nhiều module con mà mỗi module con lại là một FSMĐ có control path và data path của riêng nó. Chương này tuy không đi sâu vào trình bày từng module con có kiến trúc FSMĐ cụ thể như thế nào, nhưng đã trình bày tổng quan về chức năng của từng module con, người đọc vẫn dễ dàng hình dung ra được hoạt động của hệ thống.

Bên cạnh mã nguồn RTL thì thiết kế còn có mã nguồn testbench, đây là một testbench có nhiều tầng, viết bằng ngôn ngữ SystemVerilog theo kiểu hướng đối tượng. Quá trình kiểm chứng thiết kế đã đạt được độ bao phủ chức năng là 100%.

KẾT LUẬN

Trong báo cáo này, em đã trình bày về các vấn đề cơ bản nhất của Neural Network và FPGA. Từ việc triển khai Neural Network trên Matlab, em đã triển khai thành công Neural Network trên FPGA để nhận dạng chữ số viết tay. Kết quả thu được là độ chính xác đạt 94.02% đối với tập kiểm tra MNIST. Thiết kế này tuy chưa thể áp dụng vào một lĩnh vực cụ thể của cuộc sống, cũng như chưa thể tạo ra giá trị thương mại nhưng đó đã là một khoảng thời gian mà em học tập được nhiều về Neural Network cũng như FPGA. Đó cũng là khoảng thời gian để bản thân em cải thiện nhiều về kỹ năng sử dụng ngôn ngữ SystemVerilog, kỹ năng thiết kế phần cứng và kỹ năng sử dụng phần mềm QuestaSim trên hệ điều hành Centos.

Tuy đạt được mục tiêu đề ra, nhưng do hạn chế về mặt thời gian thực hiện, đồ án vẫn chưa phát triển được hết các tính năng của Neural Network, cụ thể là mới triển khai được việc nhận dạng, còn việc huấn luyện Neural Network thì vẫn chưa được triển khai. Trong tương lai, nếu có cơ hội, em sẽ tiếp tục nghiên cứu và phát triển các loại Neural Network trên FPGA với nhiều tính năng hơn.

TÀI LIỆU THAM KHẢO

- [1] David Kriesel (2005, May 27). *A brief introduction to neural network*. [Online]. Available : http://www.dkriesel.com/en/science/neural_networks.
- [2] http://www.lsi-contest.com/2018/shiyou_3e.html, truy nhập cuối cùng ngày 29/4/2019.
- [3] Vũ Hữu Tiệp (2018, March 27). *Machine learning cơ bản*. [Online]. Available : <https://machinelearningcoban.com/ebook/>.
- [4] Clive Maxfield, *The design warrior's guide to FPGAs*. Elsevier's Science and Technology Right Department, Oxford, UK, 2004.
- [5] Stuart Sutherland, Simon Davidmann, Peter Flake, *SystemVerilog For Design, Second Edition*. Springer Science+Business Media, LLC, 233 Spring Street, New York, NY 10013, USA, 2006.
- [6] Pong P.Chu, *FPGA prototyping by verilog examples (Xilinx SpartanTM-3 Version)*. A John Wiley & Sons, 111 River Sheet, Hoboken, 2008.
- [7] Chris Spear, *SystemVerilog for verification (Second Edition)*. Springer Science+Business Media, LLC, 233 Spring Street, New York, NY 10013, USA, 2006.
- [8] <http://yann.lecun.com/exdb/mnist/>, truy nhập cuối cùng ngày 13/05/2019.
- [9] https://en.wikipedia.org/wiki/Single-precision_floating-point_format, truy nhập cuối cùng ngày 13/05/2019.

BẢNG ĐỐI CHIẾU THUẬT NGỮ ANH VIỆT

Thuật ngữ tiếng Anh	Thuật ngữ tiếng Việt
Artificial Neural Network	Mạng neuron nhân tạo
Neural Network	Mạng neuron (nhân tạo)
Latency	Trễ xử lý
Input Layer	Lớp đầu vào
Hidden Layer	Lớp ẩn
Output Layer	Lớp đầu ra
Supervisor	Giám sát viên
Weight	Trọng số
Bias	Hạng tử tự do
Activate function	Hàm kích hoạt
Square error function	Hàm lỗi bình phương
Cost function	Hàm giá
Backpropagation	Lan truyền ngược
Learning rate	Tốc độ học
Local minimum	Cực tiểu cục bộ
Schematic-based flow	Luồng thiết kế dựa trên sơ đồ nguyên lý
HDL-based flow	Luồng thiết kế dựa trên HDL
Verification	Kiểm chứng thiết kế
Verification Plan	Kế hoạch kiểm chứng
Specification	Yêu cầu kỹ thuật
Functional Coverage	Bao phủ chức năng

Grayscale	Ảnh xám
Pixel	Điểm ảnh
Train set	Bộ huấn luyện
Validation set	Bộ xác nhận
Test set	Bộ kiểm tra
Compile	Biên dịch